INFORMATION SYSTEMS 3E

INSY7315

Group Members
ST10384670- Zanenhlanhla Tshenolo Konjwayo
ST10274490 – Amahle Ndlovu
ST10444931 – Sandiswa Ashraf Memela
ST10450938 –Siyabonga Ndlangisa
ST10077892 – Nonjabulo Mathenjwa
ST10400793 – Ayabonga Nzuza

Client - Sisterlove

TABLE OF CONTENTS

9.1.4. INTRODUCTION 2

9.1.5. REQUIREMENTS 4

9.1.5. USER REQUIREMENTS 7

9.1.7. USER STORIES 13

9.1.8. NON-FUNCTIONAL REQUIREMENTS 17

9.1.9 ANALYSIS ARTIFACTS 18

9.1.10. **IMPLEMENTATION** 22

9.1.11. DATA SCHEMA 36

9.1.12. ARCHITECTURE ARTIFACTS 47

9.1.13. SECURITY 6

9.1.14. DEVOP 7

9.1.15. RUNNING COSTS 16

9.1.16. CHANGE MANAGEMENT 19

9.1.17. APPENDICES 24

REFERENCE 26

9.1.4. INTRODUCTION

This report is submitted to the management of SisterLove International South Africa (SLiSA) to present and justify the website system developed on behalf of the organisation. It summarises the rationale behind the project, the approach taken by the development team, and the key considerations, structural, analytical and security related, that shaped the final solution. The purpose of this document is to give management confidence that the system meets SLiSA's operational needs, reflects the organisation accurately to its stakeholders, and has been built with due regard to sound software engineering and security practice.

SLiSA is a locally led, community-based organisation working in sexual and reproductive health (including HIV and PrEP awareness), gender-based violence and femicide (GBVF) prevention, and youth engagement through sport. Although historically connected to SisterLove, Inc. in the United States, SLiSA is autonomously managed and locally driven. The organisation currently operates without a permanent physical office or full-time staff, relying instead on volunteers and two consultants who coordinate activities remotely. Given these operating realities, and the financial constraints affecting the organisation, the website was identified as a strategic tool to establish credibility, attract donors and partners, support volunteer recruitment, and tell SLiSA's story independently of its US counterpart.

The system developed is a custom web application built on ASP.NET MVC (C#) with a SQL Server database, covering the pages agreed with SLiSA management - Home, About Us, Our Work, Partnerships, Previous & Legacy Work, Get Involved, Funding & Support, and Contact, together with an administrator-facing content management area, a volunteer registration workflow, and a funding/donor enquiry workflow. A custom-built application was chosen over an off-the-shelf CMS in order to give the team full, demonstrable control over authentication, data validation and security implementation, which is the focus of the PRSE module.

Management should note one trade-off arising from this decision - the brief requested a solution with minimal to no ongoing hosting or maintenance cost that non-technical volunteers could update directly. A custom ASP.NET application carries a higher

hosting and maintenance overhead than a managed platform such as WordPress, since content changes and future feature work will require developer involvement rather than being editable directly by volunteers through a simple dashboard. To keep this cost as low as practically possible, the team recommends deploying to a low-cost managed tier (for example, an entry-level Azure App Service plan with a free-tier or Basic SQL Database) and scoping a lightweight admin dashboard so that day-to-day content edits can still be performed by the consultants without developer support. This trade-off, and the reasoning behind it, should be discussed with SLiSA before final sign-off.

9.1.5. REQUIREMENTS

Introduction to Requirements Analysis

The client's website development brief, an examination of the current Sister Love Inc. website, and research into nonprofit website best practices were the sources of the specifications for the Sister Love International South Africa (SLiSA) website. The major goal is to create a polished, responsive WordPress website that builds trust with partners and funders while staying within the organization's budgetary limits. Affordability is a crucial non-functional criterion along with usability, security, accessibility, and scalability because the client specifically asked for a solution with little to no continuing hosting and maintenance expenses.

Functional Requirements

ID	Requirement	Priority	Justification
FR1	A homepage introducing Sister Love International South Africa will be provided by the system.	High	Need to establish the identity and legitimacy of the organization.
FR2	Information about the organization, mission, vision, and history will be included on the website.	High	Makes the organization more understandable to visitors and possible donors.
FR3	Programs and projects like GBVF initiatives, HIV/PReP awareness, youth	High	Essential organizational tasks that the client has asked.

	athletic programs, and community partnerships will be highlighted on the website.		
FR4	There will be a contact form on the website.	High	Facilitates communication with the community and stakeholders
FR5	A volunteer registration form will be available on the website.	High	Supports volunteer
FR6	A financial inquiry form for partners and contributors will be available on the website.	High	Supports fundraising
FR7	WordPress will make it simple for administrators to change the website's content.	High	No permanent technical staff are available.
FR8	The client's images and videos will be shown on the website.	Medium	Increases involvement and shows the impact on the organization.

FR9	Future online donation integration will be supported by the website.	Medium	Asked to be implemented in the future.
FR10	Links to social media will be included on the website.	Medium	Enhances outreach and communication.

9.1.5. USER REQUIREMENTS

According to the client brief, the following are the main users:

Donors

Require access to:

- Credibility of the organization
- Prior initiatives
- Funding inquiries
- Reports
- Contact information

Volunteers

Need access to:

- Volunteer opportunities
- Registration forms
- Upcoming projects
- Contact information

Partners with the Government and NGOs

Require access to:

- Profile of the organization
- Partnerships that benefit the community
- Details of the project
- Contact information

Administrator of the Website

Required to:

- Post news

- Upload pictures
- Modify pages
- Revise projects
- Moderate requests for contact

These duties are made possible by WordPress, which eliminates the need for programming expertise.

Research Performed on Related Organizations

In order to comprehend the structure, navigation, design language, and information architecture of the SisterLove Inc. website (USA), the team conducted an analysis.

Important conclusions include:

- A tidy and expert nonprofit design
- A strong focus on narrative
- Calls to action that are prominent
- Mobile-friendly design
- Navigation that is accessible
- Unambiguous classification of programs

The South African website will incorporate local information, programs, and community effect while adhering to these design guidelines.

Research on Target Users

Research shows that South Africa has a high percentage of internet users accessing websites through smartphones.

Therefore, the website should prioritise:

- Responsive design
- Fast loading pages
- Simple navigation
- Minimal bandwidth usage

Research on the Selection of Technology

WordPress was chosen since it provides:

- No cost for license (open source)
- large ecosystem of plugins
- Simple handling of content
- Low development expenses
- vast community of support
- Ideal for charitable organizations

Because they need more technical know-how and maintenance work, alternative CMS platforms like Drupal and Joomla were taken into consideration but ultimately rejected.

Why use WordPress?

WordPress is the most used content management system in the world, powering over 43% of all websites globally. Because of its popularity, charity organizations run by volunteers can benefit greatly from its good documentation, plugin availability, security upgrades, and ease of use.

91.6. USER ROLES

Four main user roles - Donor, Volunteer, Government/NGO Partner, and Website Administrator - have been identified for the SLiSA website based on the specifications found in the client brief. The various ways that stakeholders will engage with the website and the data or features they need are the basis for these roles.

Donor

Individuals, organizations, funders, and other stakeholders that wish to contribute financially to SisterLove International South Africa are represented by the Donor role. Donors need access to data that demonstrates SLiSA's validity and reliability, such as the organization's background, past projects, ongoing activities, community effect, and reports that are readily available.

Additionally, the website should allow donors to submit a fundraising inquiry and view funding-related information. In order to engage with SLiSA about possible financing opportunities or partnerships, they ought to have access to the organization's contact details.

Donors don't need administrative access because their main interaction will be with the website that is visible to the public. The client brief does not specify that donors require individual accounts or customized dashboards, thus this strategy seems sense.

Volunteer

Those who would like to donate their time and expertise to SLiSA's programs and community projects are represented by the Volunteer role. Volunteers need to be informed about the organization's priorities, forthcoming programs, and volunteer opportunities.

Volunteers should be able to show their interest in helping by filling out a registration form on the website. For further questions, they ought to have access to the organization's contact details.

Volunteers do not need access to the WordPress administration area, just like donations do. Viewing information that is accessible to the public and filling out the relevant online forms will be their main interactions with the system.

Government and NGO Partners

Government departments, non-governmental organizations, civil society organizations, and other stakeholders who might work with SLiSA are represented by the Government/NGO Partner role.

These users need access to comprehensive data regarding SLiSA's organizational profile, programs, past and present initiatives, community partnerships, and the results of its work. Because the website aims to build credibility and draw collaborations and financing, this information is very crucial.

Additionally, partners should be able to get in touch with SLiSA about possible collaboration by using the contact or financing inquiry feature. These stakeholders do not need access to the management features of the website, much like funders and volunteers do.

Website Administrator

The main authenticated user in charge of overseeing and maintaining the SLiSA website is the Website Administrator. Because the administrator needs access to the WordPress dashboard, this function differs greatly from the other user profiles.

The administrator will be in charge of managing inquiries made using the contact, volunteer, and fundraising forms; publishing and updating news; editing website pages; uploading photos and videos; and updating project information.

Website owners may manage what authenticated users can and cannot do thanks to WordPress's built-in role and capability system. Pages, posts, media, plugins, themes, and users can all be managed by the Administrator role in a single WordPress installation (WordPress.org, 2026).

Therefore, administrator access to the SLiSA website should be limited to those who are authorized and in charge of its upkeep. Because the administrator can change published material and control crucial website settings, this is very crucial.

Implementing WordPress Roles

The system does not need to create four distinct WordPress login roles even though four user groups have been recognized.

Rather than being authorized WordPress roles, the Donor, Volunteer, and Government/NGO Partner groups are essentially website stakeholder roles. They can use the appropriate forms to communicate with SLiSA and visit the public website without having to register.

On the other hand, the Website Administrator ought to own a verified WordPress Administrator account. WordPress offers a number of predefined roles, including as Administrator, Editor, Author, Contributor, and Subscriber, each of which has a unique set of powers (WordPress.org, 2026).

This strategy also adheres to the least privilege principle, which states that users should only be granted the permissions required to carry out their duties. WordPress advises employing roles and capabilities to limit what users may do on a website (WordPress Developer Resources, 2023).

9.1.7. USER STORIES

A. Primary Audiences

1. Potential Funders & Donors

This audience needs to quickly see credibility, alignment, and impact to decide whether to support SLiSA.

User Needs & Expectations

I. Mission Visibility: Potential funders require immediate visibility of the organisation's mission and key focus areas on the homepage to assess alignment with funding priorities.
II. Funding Information Access: Potential donors need clear access to a dedicated Funding & Support page to understand current organisational needs and the impact of contributions.
III. Programme Track Record: Philanthropic organisations require transparent documentation of past programmes, e.g. the DREAMS programme, to evaluate SLiSA's capacity to manage large-scale initiatives.
IV. Partnership Transparency: Grant-makers need to view a list of current partners, e.g. Sonke Gender Justice and the Department of Sport and Culture, to assess the strength of SLiSA's networks.
V. Engagement Pathways: Corporate Social Investment managers require a visible Contact Us form and a Partner with us call-to-action to initiate discussions on collaboration or sponsorship.
VI. Values Alignment: Potential funders need access to the About Us section to review SLiSA's core values, e.g. Community Leadership and Gender Justice, to ensure alignment with their organisational mission.

2. Government Departments

User Needs & Expectations

I. Public Sector Engagement Record: Government department officials require a clear record of events and activities involving government stakeholders, e.g. the MEC, to evaluate the organisation's level of engagement with the public sector.

II. Collaboration Pathways: Government programme coordinators need access to a dedicated Partnerships & Stakeholders page to identify potential opportunities for future collaboration with SLiSA.

3. Civil Society Organisations (CSOs) & Development Partners

As a leader of a local CSO, I want access to a clear Stakeholders page, so I can identify other organisations they collaborate with and explore opportunities for joint programming.

I. As a development partner, I want to view photos and videos from past events and activities, so I can gain a tangible sense of the organisation's reach and impact.
II. As a representative of a NGO, I want to read the organisational history, so I can understand their local legacy and how they are affiliated with the broader SisterLove South Africa network.

B. Secondary Audiences

4. Young People

User Needs & Expectations

I. Representation: Young people require access to images and videos of Youth Empowerment Through Sport events to feel inspired and reflected in SLiSA's work.
II. Health Information Access: Young adults need accessible information on HIV prevention and sexual and reproductive health to make informed decisions for themselves and their peers. Volunteer Engagement: Youth volunteers require a clear
III. Volunteer call-to-action to easily register for community events and activities.

5. Volunteers

User Needs & Expectations

I. Engagement Pathways: Potential volunteers require a dedicated Get Involved section to explore opportunities to contribute time and skills across areas such as events, communications, and administration.

II. Community Connection: Former volunteers need access to photos and videos from past events to maintain connection to the community and recognize collective impact.

6. Women and Communities Affected by HIV, GBVF and SRH challenges

User Needs & Expectations

I. Mission Alignment: Women affected by HIV need to see that the organisation's mission prioritizes addressing HIV impacts and promoting SRH rights, to confirm SLiSA as a supportive ally.
II. Issue Visibility: Community members affected by GBVF need GBVF prevention to be visibly integrated in the Our Work section to ensure the organisation is actively addressing this challenge.

7. Researchers and Stduents

This audience needs credible information and clear context.

I. Access historical data: Browse the Previous & Legacy Work section to understand SLiSA's long-term programmes and impact on HIV/SRH in South Africa.
II. Find academic material: Get clear, structured info on mission, vision, and values to use SLiSA as a case study.

8. Media and Public

This audience needs speed, clarity, and ease of use.

I. Get facts fast: Find a concise overview of Our Work and Partnerships to source accurate background for stories.
II. Browse with ease: Experience a clean, mobile-friendly site with simple navigation to learn about SisterLove SA and how to support.

Application of User Stories in Prototyping and Change Management

A. High-Fidelity Prototyping

User stories should guide design, testing, and validation:

a) Audience Prioritization: Begin with stories for primary stakeholders, including Funders, and Government Departments, as their engagement is critical to site success.
b) Journey Validation: Use stories as functional checklists to test navigation and task completion. Example: Confirm that Potential Funders can easily locate the Funding & Support page and that content addresses impact.
c) Content Alignment: Map content to user expectations identified in stories. Example: The need for "a clear track record" informs inclusion of comprehensive programme history in Previous & Legacy Work.
d) CTA Optimization: Ensure core calls-to-action - Learn About Our Work, Partner with Us, Support Our Work, Get Involved, Contact Us - are positioned and designed to meet priority user needs.

B. Change Management

User stories support internal adoption and external communication:

a) Stakeholder Engagement: Reference stories to rationalize design decisions during prototype reviews. Example: "The Funding & Support page is prioritized in navigation to meet funder objectives."
b) Staff Training: Use stories to demonstrate intended site use. Example: Position the Partnerships page as a strategic resource for engagement, not only a directory.
c) Content Development: Use stories to inform content strategy. Ensure that copy and visuals in Our Work directly address the information needs of audiences such as Government Officials and Community Leaders.

9.1.8. NON-FUNCTIONAL REQUIREMENTS

Non-functional Requirements

ID	Requirements	Justification
NFR1	The website must be responsive to mobile devices.	Smartphones are the primary means of website access for many South Africans.
NFR2	Under typical network conditions, pages should load in less than three seconds.	Enhances donor engagement and user experience.
NFR3	HTTPS encryption will be used on the website.	Safeguards user data entered forms.
NFR4	When feasible, the website must adhere to WCAG 2.1 accessibility criteria.	Guarantees that users with disabilities can access it.
NFR5	At least 99.9% of the time, the website must be accessible.	Guarantees partners' and donors' ongoing access.
NFR6	It should be simple for non-technical volunteers to manage the website.	Client has no permanent IT staff.
NFR7	The website will minimize ongoing operating expenses.	Explicit client request because of financial limitations.
NFR8	The website must be expandable to handle future expansion.	The organization anticipates more funding and partnering opportunities.

9.1.9 ANALYSIS ARTIFACTS

This section documents how the team analysed the requirements set out in SLiSA's website development brief and subsequent correspondence in order to derive a domain model for the system and presents the behavioural diagrams (sequence and state diagrams) that were produced to describe how the system realises the key use cases. The requirements were gathered primarily from two sources - the Website Development Brief provided by Ms Nomsa Mothapo (Ops and Admin Consultant, SisterLove Inc.), and the accompanying Website Content document (Mothapo, 2026). These identified the site's purpose, target audiences, organizational context, core focus areas, scope of work, and design constraints.

9.1.9.1 Requirements Summary

From the brief, the following functional and non-functional requirements were extracted as the basis for analysis

- The site must present SLiSA as a locally led, independently managed organization, with any SisterLove Inc. affiliation limited to background or legacy sections.
- Content must be organized into eight core pages: Home, About Us, Our Work, Partnerships, Previous & Legacy Work, Get Involved, Funding & Support, and Contact.
- The system must support contact, volunteer, and funding enquiry forms, with a phased approach to future donation integration.
- The platform must be mobile-friendly, low-cost to host and maintain, and manageable by non-technical volunteers/consultants.
- The design must be professional, warm, dignified and survivor-respectful, visually aligned with the SisterLove Inc. reference site while remaining South Africa-centred.
- Admins must be able to log in securely and manage website content without direct database access

9.1.9.2 Domain Model

The domain model below represents the key entities identified from the requirements and the relationships between them. It reflects the content structure of the website (pages, focus areas, media) as well as the interactions the site must support (enquiries, volunteering, partnerships).

Domain Entity Summary

Entity	Description / Attributes	Relationships
Administrator	An authenticated staff/consultant user managed via the Identity Service: username, hashed password, role, last login date.	Logs in via Authentication Controller; creates/edits ContentPage records
Content Page	A publishable page: title, slug, body content, featured media, publish status, last-updated date.	Managed by Administrator via Content Controller/Content Service; stored in SQL Server Database
VolunteerRegistration	A submitted registration record: submission date, status (open/complete/submitted/reviewed), linked Volunteer.	Progresses through the Volunteer Registration state diagram (Figure 5)
Volunteer	A prospective volunteer: name, email, phone, area of interest, registration status.	Submits a VolunteerRegistration via Volunteer Controller/Volunteer Service
FundingPartner (Donor)	An organisation or individual submitting a funding enquiry: name, email, organisation, message.	Submits a FundingEnquiry via Funding Controller/Funding Service
FundingEnquiry	A submitted funding/donor enquiry record: submission date, message, status.	Routed to Administrator for follow-up; stored in SQL Server Database

Common Vulnerabiities and Threats

This table below displays the risks most relevant to a publicly accessible system like this, following the injection and cross-site scripting categories described in the OWASP Top 10 (OWASP, 2021), together with the likelihood/impact rating and mitigation applied for each

Risk Register

Risk	Description	Likelihood/Impact	Mitigation Strategy
Cross-Site Scripting (XSS)	Malicious script injected via form fields (contact, volunteer, funding enquiries) and later rendered in the administrator dashboard or on the public site.	High / High	Razor views HTML-encode output by default; all Data Transfer Objects use Data Annotations for server-side input validation; any content requiring rich text is passed through a whitelist-based HTML sanitizer before storage; a Content-Security-Policy header restricts script

			execution to trusted sources.
SQL Injection	Attempt to manipulate database queries through form or URL input.	Medium / High	All data access uses Entity Framework LINQ queries or fully parameterised ADO.NET commands; no SQL is built by concatenating user input; the application's database account has least-privilege permissions (no DDL rights).
Cross-Site Request Forgery (CSRF)	A malicious site tricks a logged-in administrator into performing an unintended action, e.g. editing or deleting content.	Low / Medium	ASP.NET MVC's AntiForgeryToken is issued on every form and validated with [ValidateAntiForgeryToken] on all state-changing Content Controller actions.
Brute-force / weak credentials	Automated guessing of administrator login credentials given the site has few active editors.	Medium / High	ASP.NET Identity hashes passwords with PBKDF2, enforces a strong password policy, and locks an account out after repeated failed attempts; two-factor authentication is enabled for administrator accounts.
Outdated Dependencies	Known vulnerabilities in unpatched .NET runtime or NuGet packages.	Medium / High	NuGet packages are kept to a minimal, actively maintained set; a maintenance schedule is documented for applying .NET and package security updates.
Spam / abuse of enquiry forms	Automated bots submitting spam or malicious payloads through the Volunteer Registration and Funding Enquiry forms.	High / Low–Medium	A CAPTCHA/honeypot mechanism and server-side rate limiting are applied to all public forms in addition to the model validation and sanitisation described above.
Data loss	Content or enquiry data lost due to hosting failure, given SLiSA has no dedicated IT support.	Low / High	Automated, scheduled SQL Server database backups are configured through the hosting provider (e.g. Azure SQL automated backups), ensuring recoverability without manual intervention.

Cross Site Scripting XSS in detail

Because the system accepts free-text input from unauthenticated visitors through the Volunteer Registration and Funding Enquiry forms XSS was treated as the highest-priority risk, consistent with its classification under the injection category of the OWASP (OWASP, 2021). Stored XSS was of particular concern, since a payload submitted through a form could be persisted in the SQL Server Database and later rendered in the Administrator's dashboard when enquiries are reviewed. The following controls were applied together, following the principle that input validation alone is insufficient and must be paired with output encoding

- Input validation - Data Annotation attributes on each model (e.g. [EmailAddress], [StringLength]) reject malformed input at the Controller before it reaches the Service layer.
- Sanitisation on input - any field that could contain markup is passed through a whitelist HTML sanitiser before being persisted to the database, stripping or neutralising script tags.
- Output encoding on render - Razor's default HTML encoding ensures all user-supplied content is escaped for its context (HTML body, attribute, or URL) wherever it is displayed, on the public site or in the Administrator Dashboard, so any characters that slipped through are rendered inert rather than executed.
- Content Security Policy - a restrictive CSP header limits script execution to the application's own domain, reducing the impact of any payload not caught by sanitisation.
- Least privilege for roles - only the Administrator role can submit content through the rich-text editor in the Content Controller; Volunteer and Funding Partner submissions are always treated as plain text.

9.1.10. IMPLEMENTATION

Behavioural Diagrams - Sequence Diagrams

The following sequence diagrams describe how the four primary use cases identified above are realised across the system's layers (boundary → controller → service → database), consistent with the entities in the domain model.

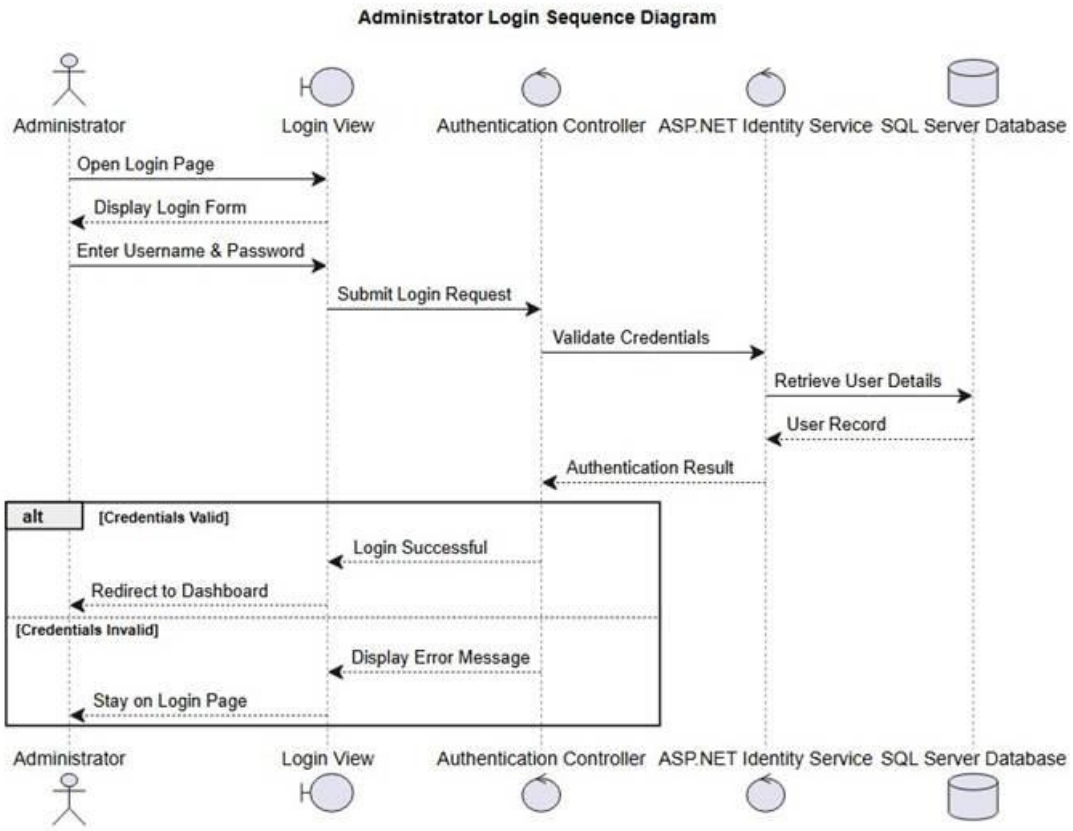


Figure 1: Administrator Login Sequence Diagram

draw.io, 2026. diagrams.net [online]. Available at:

https://drive.google.com/file/d/15cOUuT0rqCfYYYwo8rTMxhOy8Z8lAKR_/view?usp=sharing [Accessed 10 August 2026].

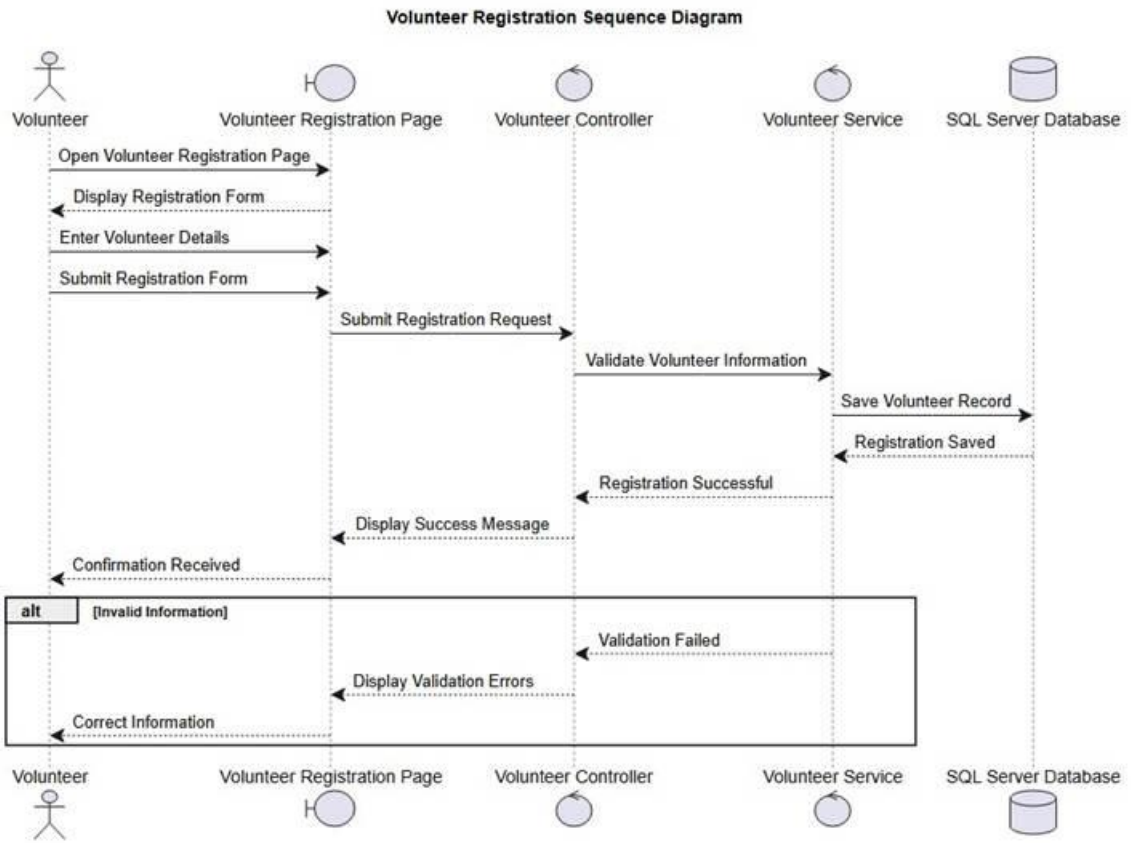
Done by Siyabonga

UML Sequence Diagram – Administrator Login

The Administrator Login sequence diagram illustrates the interaction between the administrator and the SisterLove website during the authentication process. The process begins when the administrator opens the login page and enters their username and password. The Login View sends the submitted credentials to the Authentication Controller, which passes the information to the ASP.NET Identity Service for validation.

The Identity Service communicates with the SQL Server database to retrieve the administrator's stored account information. The returned information is then used to determine whether the credentials are valid. If authentication is successful, the administrator is redirected to the administration dashboard. If the credentials are incorrect, the system displays an error message and allows the administrator to remain on the login page and try again.

This sequence demonstrates how authentication is handled securely through the application layers and ensures that only authorised administrators can access protected website management functionality.



Z

Figure 2 — Volunteer Registration Sequence Diagram

draw.io, 2026. diagrams.net [online]. Available at:

https://drive.google.com/file/d/1L1YPAUh3ieXrdl38ysmmZ9kK7A77gDc_/view?usp=sharing [Accessed 10 August 2026].

UML Sequence Diagram – Volunteer Registration

The Volunteer Registration sequence diagram illustrates how a community member or potential volunteer registers their interest through the SisterLove website. The process starts when the volunteer opens the registration page and completes the required information in the registration form.

After the form is submitted, the request is passed to the Volunteer Controller, which communicates with the Volunteer Service to validate the submitted information. If the information is valid, the volunteer's details are stored in the SQL Server database. The

system then returns a successful response and displays a confirmation message to the volunteer.

The diagram also represents an alternative flow for invalid information. If validation fails, the system returns validation errors to the registration page, allowing the volunteer to correct the information and resubmit the form.

This process supports one of the client's key requirements by providing a structured method for community members to express their interest in volunteering while ensuring that submitted information is validated before being store

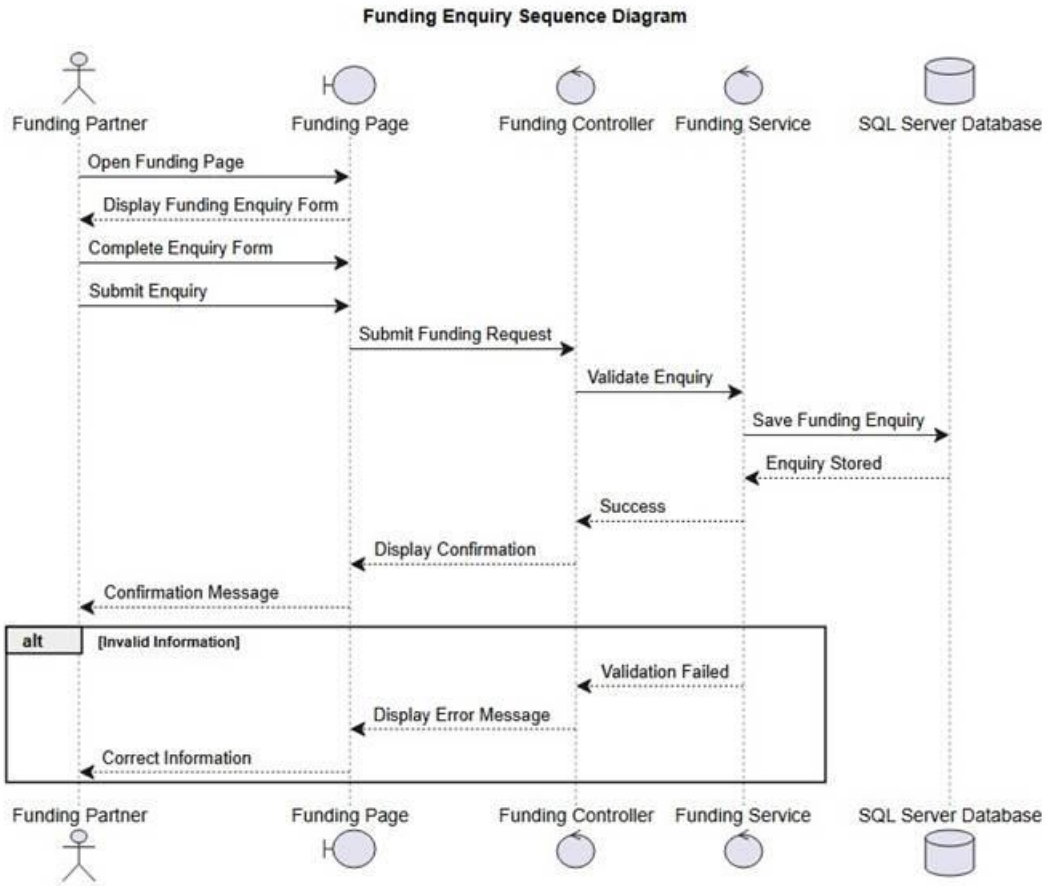


Figure 3 - Funding Enquiry Sequence Diagram

draw.io, 2026. diagrams.net [online]. Available at:

https://drive.google.com/file/d/1nJ5WKPBvkLtArCokFpLYVuKfNMbELZAo/view?usp=sharing [Accessed 10 August 2026].

Done by Siyabonga

UML Sequence Diagram – Funding Enquiry Submission

The Funding Enquiry sequence diagram illustrates the process through which potential funders, donors, or funding partners can submit an enquiry to SisterLove South Africa. The process begins when the user accesses the Funding page and completes the funding enquiry form.

The submitted information is passed from the Funding Page to the Funding Controller, which communicates with the Funding Service to validate the enquiry. Once the information has been successfully validated, the enquiry is stored in the SQL Server database. The system then returns a successful response and displays a confirmation message to the user.

If required information is missing or invalid, the system displays an error message and allows the user to correct the submitted information.

This sequence supports the client's requirement for a funding enquiry facility and contributes to the organisation's objective of attracting funders, donors, and strategic partners.

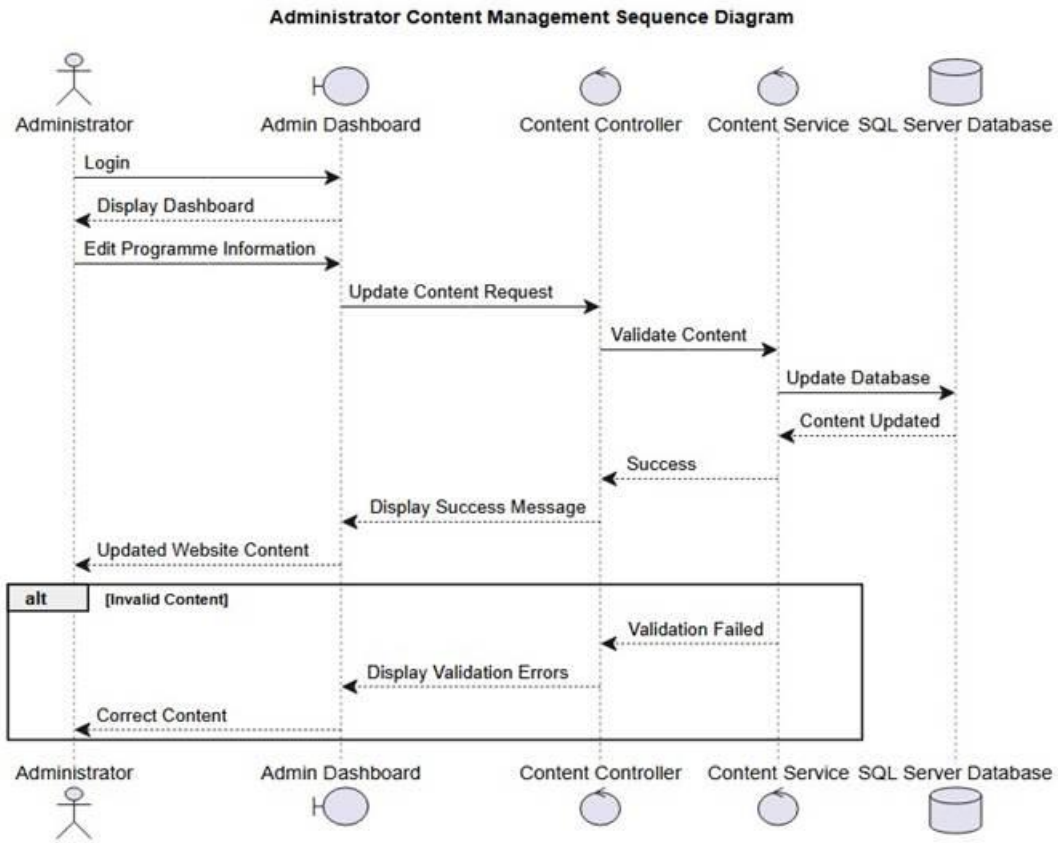


Figure 4 — Administrator Content Management Sequence Diagram

UML Sequence Diagram – Administrator Content Management

draw.io, 2026. diagrams.net [online]. Available at:

https://drive.google.com/file/d/1uuKH93s4wKLua_LKjlvwwaDPlonw8cb4/view?usp=sharing [Accessed 10 August 2026].

Done by Siyabonga

The Administrator Content Management sequence diagram illustrates how authorised administrators will manage and update information displayed on the SisterLove South Africa website. This includes information such as programmes, partnerships, previous work, organisational information, and other website content.

The administrator accesses the administration dashboard and selects the content that needs to be updated. The dashboard sends the update request to the Content Controller, which communicates with the Content Service to validate the information. Once validation is successful, the updated content is stored in the SQL Server database.

The system then returns a successful response and provides confirmation that the content has been updated. If the submitted content does not meet the required validation rules, the system displays validation errors and allows the administrator to make corrections.

This sequence is particularly important because the client requested a website that is easy to update. It allows authorised users to maintain accurate and current information without requiring changes directly to the application's source code.

Behavioural Diagrams: State Diagrams

The state diagrams below describe the lifecycle of the two entities in the domain model whose status changes over time: the volunteer registration process, and the website content publishing workflow.

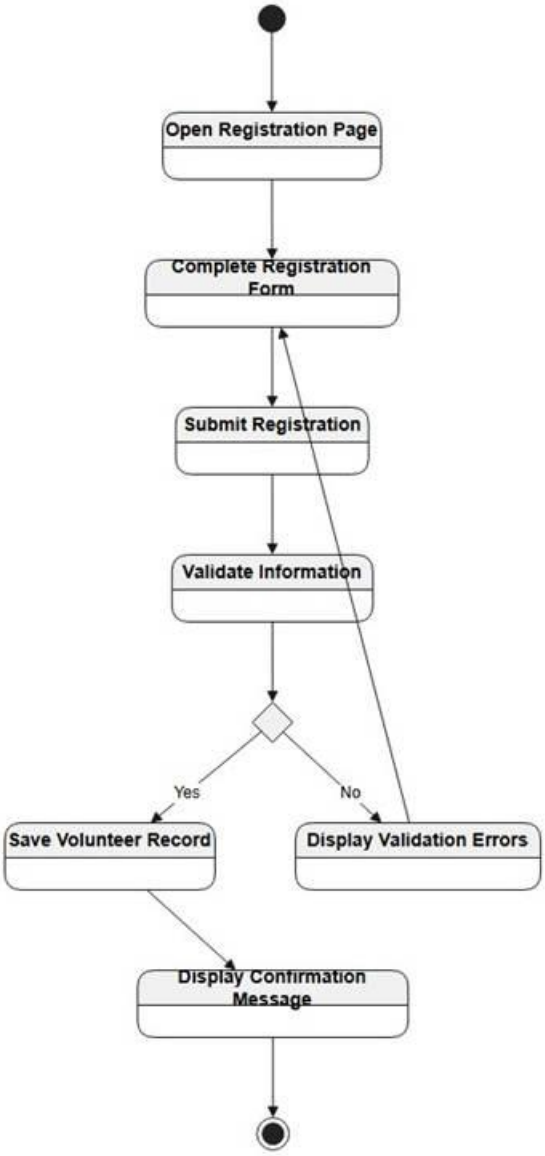


Figure 5 – Volunteer Registration State Diagram

draw.io, 2026. diagrams.net [online]. Available at: https://drive.google.com/file/d/1r6Fu_4d2Ce5C1TKty4MZNGvZh6BgkzY7/view?usp=sharing [Accessed 10 August 2026].

Done by Siyabonga

UML State Diagram – Volunteer Registration

The Volunteer Registration state diagram represents the different states that a volunteer registration passes through during its lifecycle. The process begins when the

volunteer opens the registration page and progresses to completing and submitting the registration form.

Once the form is submitted, the system enters the validation state where the supplied information is checked. If the information is invalid, the system enters the validation-error state and returns the volunteer to the registration form so that the required corrections can be made.

If the information is valid, the system moves to the Save Volunteer Record state, where the registration information is stored in the SQL Server database. The system then displays a confirmation message before reaching the final state.

The diagram provides a clear representation of both the successful and unsuccessful paths of the registration process. It demonstrates how the application handles user input and prevents invalid information from progressing through the system.

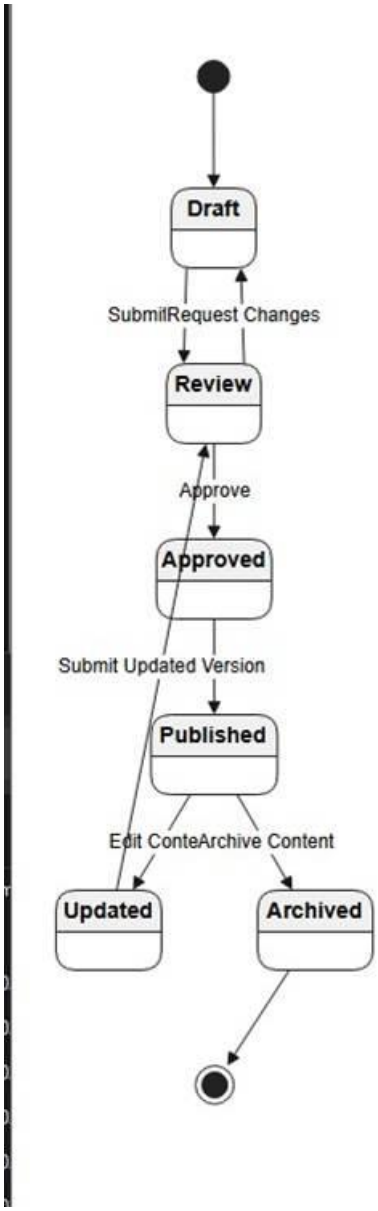


Figure 6 — Website Content Lifecycle State Diagram
draw.io, 2026. diagrams.net [online]. Available at: https://drive.google.com/file/d/14qimGKr2Fgc1gyeK92dHdWcc9EF8Ac94/view?usp=sharing [Accessed 10 August 2026].

Done by Siyabonga

UML State Diagram – Website Content Lifecycle

The Website Content Lifecycle state diagram illustrates how content within the SisterLove South Africa website can progress through different states during its lifecycle. New content initially exists in a Draft state, where an authorised administrator can create or edit the information.

Once the content is ready, it is submitted for Review. During the review stage, the content can either be approved or returned to the Draft state for further changes. Approved content can then be Published and made available to website visitors.

Published content can subsequently be Updated when information changes. Updated content returns to the review process to ensure that changes are checked before being published again. Content that is no longer relevant can be moved to the Archived state.

This lifecycle supports the client's requirement for a professional and easy-to-maintain website by establishing a controlled process for managing information. It also helps ensure that outdated or inaccurate information is not unnecessarily displayed to website visitors.

Justification of the Domain Model

The model separates content management concerns (ContentPage) from stakeholder interaction concerns (VolunteerRegistration, FundingEnquiry), each realised through its own Controller/Service pair, consistent with the layered architecture shown in the sequence diagrams. This separation mirrors the brief's two central requirements: a content-rich, credible public presence, and a set of low-friction channels for donors, partners and volunteers to engage with SLSA. The Volunteer Registration and Website Content Lifecycle state diagrams (Figures 5 and 6) further show that both workflows follow a similar submit → review → approve/publish pattern, which supports consistent handling and future extension, for example when donation integration is introduced in a later phase.

Security

Security was a central design consideration throughout development, both because the system will be publicly accessible and because it collects personal information from donors, partners and volunteers, and stores it in a SQL Server database accessed through an ASP.NET MVC application. This section presents a risk analysis of the system, discusses the security principles applied, and details the specific mitigations implemented, including cross-site scripting (XSS) prevention, in line with the security principles and techniques covered in the PRSE module.

Security Approach

The team applied a defence-in-depth strategy, layering controls at the network, platform, application and data levels rather than relying on any single safeguard. Three guiding principles from the module were applied throughout: least privilege (each user, role and service account is granted only the access it needs, e.g. the application's database account cannot alter schema), input validation and output encoding (all data crossing a trust boundary, from Boundary to Controller to Service to Database, is treated as untrusted until proven otherwise), and secure defaults (the system fails closed rather than open, and insecure configurations must be deliberately opted into rather than deliberately opted out of).

Network Security

- → HTTPS/TLS is enforced site-wide, with HTTP requests redirected to HTTPS via ASP.NET's RequireHttpsAttribute/URL Rewrite rules, protecting form submissions and the administrator login (Figure 1) from interception.
- → The administrator login and content management endpoints are reachable only over the authenticated area of the site; ASP.NET Identity's account lockout policy rate-limits repeated failed login attempts.
- → A web application firewall (WAF), for example Azure's built-in Application Gateway WAF or an equivalent on the chosen host, filters malicious traffic before it reaches the application layer.
- → The SQL Server Database is not exposed on a public port; firewall rules restrict database connections to the application server only, and the application connects using a leastprivilege service account rather than a database administrator account.

The deployment/network boundaries described above are illustrated in the architecture diagram placed at the end of this subsection, which should be read alongside the risk table in Section 3.3.

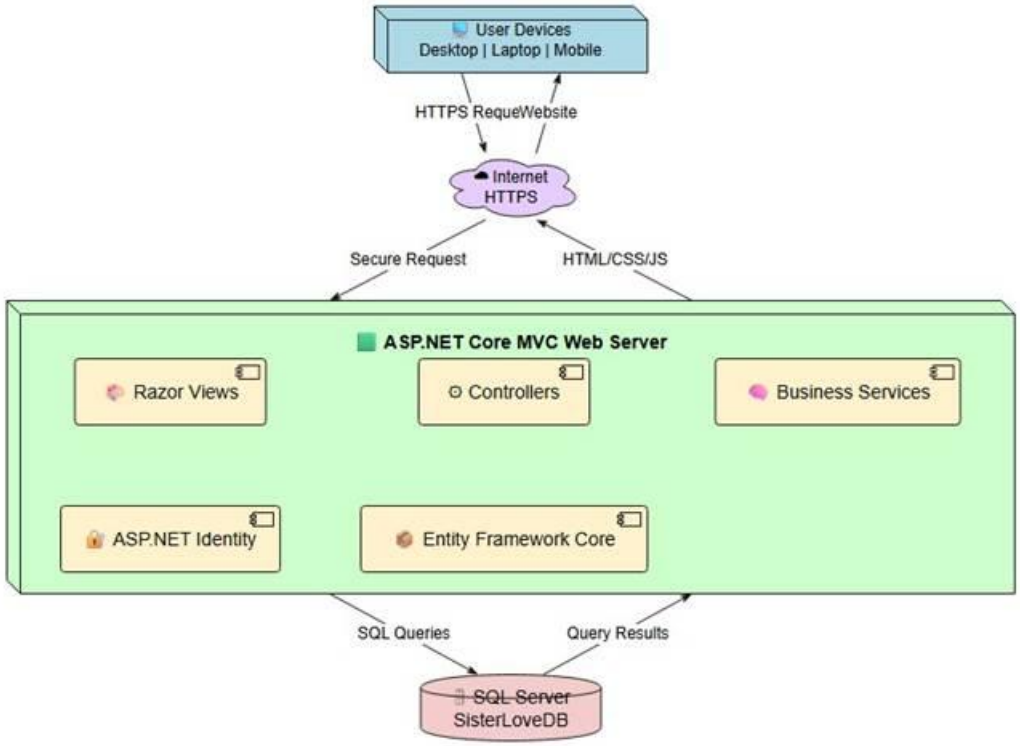


Figure 7 -Deployment Architecture Diagram

draw.io, 2026. diagrams.net [online]. Available at: https://drive.google.com/file/d/1P9KI2CcW8z9hnQK5O1Bj5FvLwD2UvM4h/view?usp=sharing [Accessed 10 August 2026].

Done by Siyabonga

UML Deployment Diagram – SisterLove Website

The UML Deployment Diagram illustrates the proposed technical environment in which the SisterLove International South Africa website will operate. Website visitors will access the application using desktop computers, laptops, tablets, or mobile devices through a web browser.

User requests will be transmitted over a secure HTTPS connection through the Internet to the ASP.NET Core MVC web application. The web application will contain the presentation layer implemented using Razor Views, controllers responsible for handling requests, business services responsible for application logic, and Entity Framework Core responsible for communication with the database.

The ASP.NET Core MVC application will communicate with the SQL Server database to store and retrieve information required by the website. This may include administrator accounts, volunteer registrations, funding enquiries, contact information, and website content.

The processed information will then be returned to the user's browser and presented as the SisterLove website. The deployment architecture separates the user interface, application logic, and data storage components, supporting maintainability, scalability, and security.

The deployment environment may initially be implemented using a suitable development or low-cost hosting environment. As the project progresses, the application can be deployed to a production hosting platform such as Microsoft Azure, depending on the client's final hosting and financial requirements.

9.1.11. DATA SCHEMA

9.1.11.1 Data Storage Technology

The SisterLove South Africa (SLiSA) web application requires a structured and secure method of storing organisational information, website content, and user-generated data. To achieve this, the project will utilise Microsoft SQL Server as the primary Relational Database Management System (RDBMS). SQL Server was selected because it integrates directly with ASP.NET Core MVC and Entity Framework Core, the technologies chosen for the development of the application, allowing efficient object-relational mapping (ORM) and simplified database management.

A relational database is particularly suitable for this project because the majority of the information managed by the website is highly structured and interconnected. Examples include registered users, volunteer applications, funding enquiries, organisational programmes, partnerships, news articles, media resources, and contact enquiries. Each of these entities shares logical relationships with other entities within the system. SQL Server enables these relationships to be enforced through primary and foreign keys, thereby maintaining referential integrity and reducing the likelihood of inconsistent or duplicate data. (Microsoft, 2025).

Another advantage of SQL Server is its support for ACID (Atomicity, Consistency, Isolation and Durability) transaction principles. These principles ensure that every transaction performed within the system is completed accurately and reliably. For example, when a volunteer submits an application, the system guarantees that all information is stored successfully before the transaction is committed. If an error occurs during the process, SQL Server automatically rolls back the transaction, preventing incomplete or corrupted records from being stored. (Microsoft, 2023).

Security is another important factor influencing the choice of SQL Server. Since the website stores user accounts and enquiry information, the database must protect sensitive data from unauthorised access. SQL Server provides authentication, authorisation, encryption support, backup and recovery mechanisms, and role-based access control. These features align with the project's requirement to provide secure

administrator authentication and protect user information from common security threats. (Microsoft, 2025; Microsoft, 2026).

Furthermore, SQL Server has been selected because it provides excellent scalability. Although the initial implementation focuses on content management, volunteer engagement, and funding enquiries, the client has indicated that future phases may introduce online donations, event management, and additional community engagement services. SQL Server allows these features to be incorporated without requiring major modifications to the existing database structure.

Finally, SQL Server is widely used within industry and higher education. Its compatibility with Microsoft development technologies, comprehensive documentation, and long-term support make it an appropriate choice for developing a professional, maintainable, and scalable solution for SisterLove International South Africa.

9.1.11.2 Database Design

The database was designed using the principles of relational database modelling, where each table represents a single business entity and relationships between entities are established through primary and foreign keys. This design approach promotes data integrity, minimises redundancy, and improves maintainability throughout the lifecycle of the application (Microsoft, 2025).

The database structure was developed directly from the functional requirements gathered during system analysis. Each entity corresponds to a real-world object or process identified within the organisation's operations. For example, registered users interact with the website by creating accounts, submitting volunteer applications, funding enquiries, or contact messages, while administrators are responsible for publishing organisational content such as programmes, partnerships, news articles, media resources, and previous work.

The use of separate entities for programmes, partnerships, legacy projects, and news articles provides greater flexibility than storing all website content in a single table. Each entity contains information relevant to its specific purpose while remaining linked

through administrator ownership and media relationships. This approach simplifies future maintenance and allows content to be updated independently without affecting unrelated sections of the website.

Media management has also been separated into its own entity. Instead of storing image and video information within multiple tables, a dedicated Media entity allows digital assets to be reused across different sections of the website while reducing duplication. This approach improves maintainability and supports efficient content management through the administrator portal.

The design also considers the future scalability of the system. Although online donations and event management are outside the scope of the initial implementation, dedicated entities have already been incorporated into the database model. Including these entities at the design stage demonstrates that the database architecture has been developed with future expansion in mind, reducing the need for structural redesign as additional functionality is introduced.

9.1.11.3 ERD

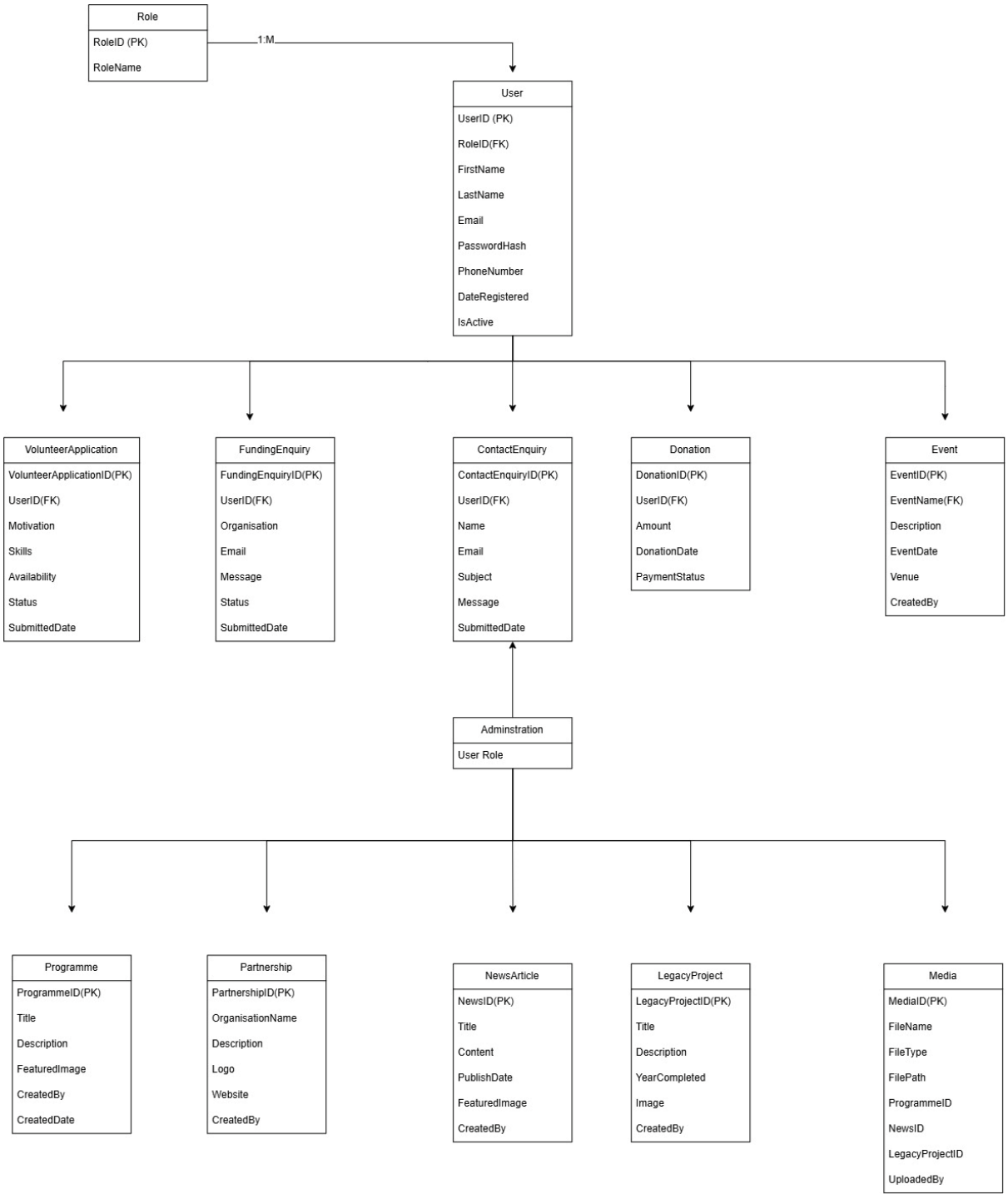


Figure 8 -Entity Related Diagram

https://drive.google.com/file/d/1PW7wTOz5Jqo0cE53N2Ncu2nMTFGClusy/view?usp=sharing

9.1.11.4 Relationship Explanation

The database has been designed using one-to-many relationships to support efficient storage and minimise redundancy.

- One **Role** may be assigned to many **Users**, while each User belongs to one Role.
- One **User** may submit multiple Volunteer Applications.
- One **User** may submit multiple Funding Enquiries.
- One **User** may submit multiple Contact Enquiries.
- One **Administrator** (represented as a user with an administrator role) may create many Programmes, Partnerships, News Articles, Legacy Projects, Events, and Media records.
- One **Programme** may have multiple Media files associated with it.
- One **News Article** may include multiple Media files.
- One **Legacy Project** may also contain multiple Media files.

These relationships ensure that information is stored once and reused where necessary, improving maintainability and reducing duplication.

9.1.11.5 Entity Descriptions

This section describes each entity included in the Entity Relationship Diagram (ERD) and explains its purpose within the SisterLove South Africa (SLiSA) website. Each entity represents a specific real-world object or process that supports the website's functionality. The separation of data into individual entities reduces redundancy, improves maintainability, and ensures that information can be managed efficiently.

Role

The **Role** entity stores the various permission levels available within the system. Role-Based Access Control (RBAC) is implemented to ensure that users are only granted access to features relevant to their responsibilities (Microsoft, 2026). For example, a

registered user can update their profile and submit enquiries, while an administrator has additional privileges to manage website content.

Separating roles into their own entity improves scalability by allowing additional roles to be introduced in future without requiring changes to the User entity.

User

The **User** entity stores the personal information and login credentials of individuals registered on the website (Microsoft, 2025). Every user is assigned a role through the Role entity, which determines the actions they are permitted to perform.

Registered users may update their profiles, submit volunteer applications, funding enquiries, and contact enquiries. Storing user information in a central entity prevents duplication, as the same user may interact with multiple areas of the system.

Programme

The **Programme** entity stores information about the organisation's programmes and initiatives, including Sexual and Reproductive Health and Rights (SRHR), HIV and PrEP awareness, Gender-Based Violence and Femicide (GBVF), Youth Engagement Through Sport, and other future community initiatives.

Each programme contains descriptive information and may have associated media files that enhance the presentation of information on the website. Administrators are responsible for creating and maintaining programme content.

Partnership

The **Partnership** entity stores details of organisations that collaborate with SisterLove South Africa. Partnerships are an important component of the organisation's work, as they demonstrate collaboration with government departments, civil society organisations, donors, and community stakeholders.

This entity allows administrators to publish information about partner organisations, including descriptions, logos, and website links where applicable.

Legacy Project

The **LegacyProject** entity stores information regarding previous projects completed by the organisation. These projects demonstrate the impact and history of SisterLove South Africa and help establish credibility with potential donors, volunteers, and community stakeholders.

Each legacy project may include descriptive information, completion dates, and associated media.

NewsArticle

The **NewsArticle** entity stores organisational announcements, educational articles, awareness campaigns, and community updates. This allows the organisation to regularly publish current information while maintaining a historical archive of previous publications.

Each news article is created and managed by an administrator and may include associated media files.

Media

The **Media** entity stores all images and videos displayed throughout the website. Instead of storing image paths within multiple entities, the Media entity centralises the storage of multimedia information.

This design promotes data consistency and allows photographs or videos to be associated with programmes, news articles, or legacy projects without unnecessary duplication.

VolunteerApplication

The **VolunteerApplication** entity stores applications submitted by users interested in volunteering for the organisation. Information such as motivation, skills, availability, and application status is recorded to support the volunteer recruitment process.

Each volunteer application belongs to a registered user, while administrators are responsible for reviewing and managing applications.

FundingEnquiry

The **FundingEnquiry** entity stores enquiries submitted by organisations or individuals interested in financially supporting SisterLove South Africa. These enquiries enable the organisation to communicate with prospective donors and establish future funding opportunities.

Each funding enquiry is linked to the registered user who submitted it.

ContactEnquiry

The **ContactEnquiry** entity stores general enquiries submitted through the website's contact page. These enquiries may include requests for information, partnership enquiries, or community support requests.

Where applicable, enquiries are linked to registered users to provide administrators with a complete interaction history.

Event (Future Enhancement)

Although event management is not included within the first implementation phase, the client indicated that future expansion may include community events, awareness campaigns, and outreach activities. The Event entity has therefore been incorporated into the database design to ensure future scalability.

Including this entity during the design stage reduces future redevelopment effort and demonstrates proactive database planning.

Donation (Future Enhancement)

The client has indicated that online donations will be implemented during a future development phase. The Donation entity has therefore been included within the database design to accommodate secure financial transactions once payment gateway integration becomes available.

Planning for future donation functionality ensures that the database can be extended without major structural modifications.

9.1.11.6 Relationship Analysis

The relationships shown in the Entity Relationship Diagram define how information is connected throughout the database. These relationships have been designed using one-to-many cardinality, ensuring that data is stored efficiently while maintaining referential integrity (Microsoft, 2025).

The relationship between the **Role** and **User** entities is one-to-many. A single role may be assigned to multiple users, while each user is assigned only one role. This approach simplifies permission management and supports role-based access control.

The **User** entity forms the central point of interaction within the system. A registered user may submit multiple volunteer applications, funding enquiries, contact enquiries and, in future, donations and event registrations. Each of these records belongs to only one user, ensuring accountability and simplifying record management.

Administrative users are responsible for managing organisational content. Consequently, one administrator may create multiple programmes, partnerships, news articles, legacy projects, media records, and future events. Associating each record with its creator provides accountability and supports audit requirements.

The **Programme**, **NewsArticle**, and **LegacyProject** entities each maintain one-to-many relationships with the **Media** entity. This design enables multiple images or videos to be associated with a single content item while avoiding duplication of multimedia resources.

The future **Donation** and **Event** entities have also been linked to the User entity. Although these modules are outside the current implementation scope, incorporating them within the database design demonstrates that the system architecture has been designed to support future organisational growth without requiring extensive redesign.

Overall, these relationships promote efficient storage, reduce redundancy, maintain data integrity, and provide a flexible foundation capable of supporting both current and future business requirements.

9.1.11.8 Database Normalisation

The proposed database has been designed according to the principles of **Third Normal Form (3NF)** to improve efficiency, reduce redundancy, and maintain data integrity.

First Normal Form (1NF)

First Normal Form requires that every table contains atomic values and that each record can be uniquely identified using a primary key. The proposed database satisfies this requirement by ensuring that each entity has a unique primary key and that attributes contain single, indivisible values. For example, each user record stores one email address and one phone number rather than multiple values within a single field.

Second Normal Form (2NF)

Second Normal Form requires that all non-key attributes are fully dependent on the primary key. Since each table uses a single-column primary key, every non-key attribute depends entirely on that primary key. Information relating to programmes, partnerships, volunteer applications, and enquiries is therefore stored within their respective entities rather than being duplicated elsewhere.

Third Normal Form (3NF)

Third Normal Form eliminates transitive dependencies by ensuring that non-key attributes depend only on the primary key and not on other non-key attributes. For example, user roles are stored separately within the Role entity instead of repeatedly storing role names within every user record. Similarly, media information is stored independently and linked through foreign keys rather than duplicated across multiple content entities.

Applying Third Normal Form improves consistency, simplifies maintenance, and enables future system expansion without unnecessary structural changes.

9.1.11.9 Summary

The proposed database schema provides a secure, scalable, and maintainable data storage solution for the SisterLove South Africa website. By implementing Microsoft SQL Server as the relational database management system, the project benefits from

strong data integrity, efficient querying, robust security, and seamless integration with ASP.NET Core MVC and Entity Framework Core.

The database has been designed around the organisation's operational requirements by supporting user registration, volunteer engagement, funding enquiries, contact management, organisational programmes, partnerships, legacy projects, news publications, and multimedia management. Furthermore, the inclusion of future entities such as **Donation** and **Event** demonstrates that the database architecture has been developed with scalability in mind.

The normalised relational design ensures that data redundancy is minimised while maintaining accurate relationships between entities through primary and foreign keys. Consequently, the proposed data schema provides a reliable foundation for the implementation of the SLiSA website and supports the organisation's long-term digital transformation objectives.

9.1.12. ARCHITECTURE ARTIFACTS

Selected Pattern: Layered (N-Tier) Architecture

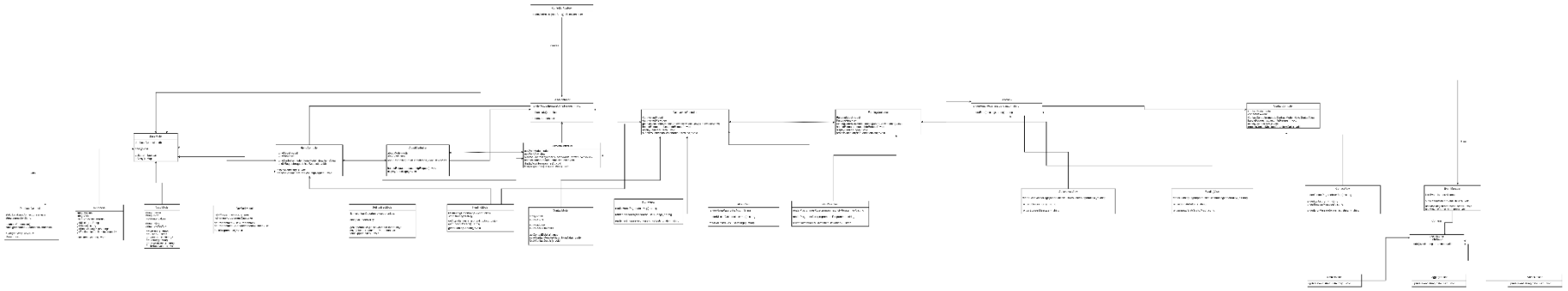
Why this Architecture?

A layered architecture fits SisterLove South Africa because it separated the UI, logic and the data for easier for updates, scales components independently, supports modular team development, reduces hosting costs through efficient resource use, and enables a mobile accessible user experience.

Other options:

- Microservices: Rejected - too complex and costly for this project's scope
- Serverless: Used only for specific functions like forms and image handling. Not adopted as the main pattern due to vendor dependency concerns.

Design Patterns – UML diagram



[Designed on Draw.io]

[Draw.io link : https://drive.google.com/file/d/1KEGgQzv-0zkr_bDVzQNjensB8AbXOO8T/view?usp=sharing

[Made by Amahle Ndlovu]

Brief:

- *MVC Architectural Pattern*

  MVC splits the app into 3 parts for cleaner, more maintainable code: Model - Data & logic. Example; AboutModel, ContactModelView - UI & presentation. e.g. HomeView, ContactView Controller - Request handling. Example; AboutController routes requests between Model and View

- *DatabaseConnection - Singleton Pattern*

We use the Singleton pattern to maintain one global database connection throughout the application lifecycle. This approach prevents multiple concurrent connections, conserves server resources, and ensures stable performance. Access is controlled via getInstance (), and all data models use this centralized connection.

- *ControllerFactory - Factory Pattern*

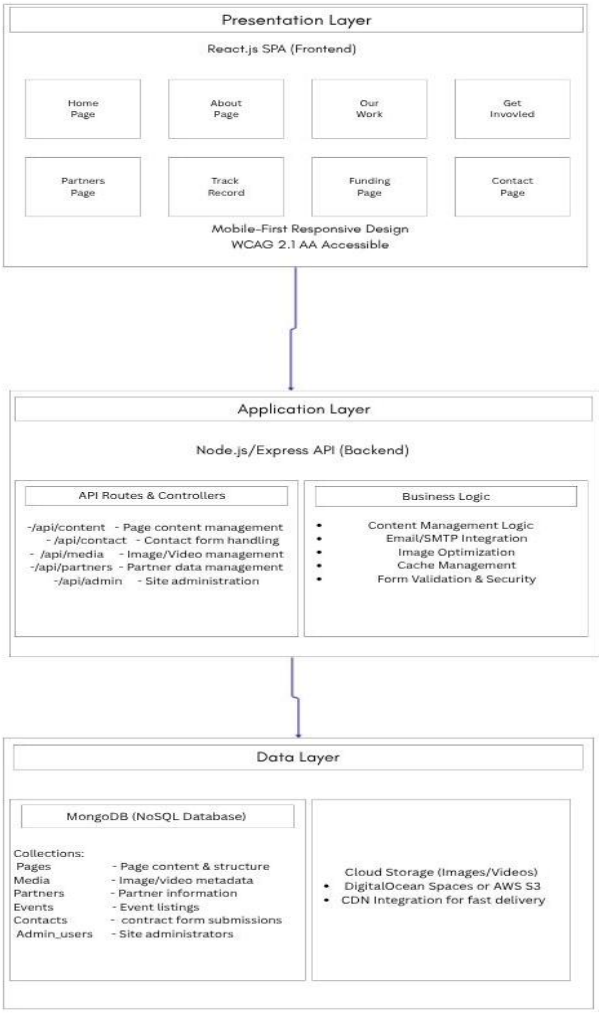
We use the Factory pattern to manage Controller instantiation. Instead of creating Controllers directly, requests are routed through ControllerFactory.createController(). Based on the page requested, it returns the appropriate Controller. And with this approach it keeps the main application flow simple, avoids code duplication, and allows new page Controllers to be added with minimal changes.

- *EventManager – Observer Pattern*

  We use the Observer pattern to handle cross-cutting tasks like notifications and logging. When an action occurs for example a contact form submission - the ContactController emits an event. The EventManager then notifies all registered listeners such as EmailNotifier, LoggingNotifier, and AdminNotifier. This keeps the main business logic independent and lets us add new responses to events without modifying existing code.

Architecture Patterns - System architecture diagram:

Figure 1



[Designed on Canva]

[Made by Amahle Ndlovu]

Canva link: https://canva.link/qi3he7vk3k7akhe

Brief: The SisterLove South Africa (SLiSA) website is architected as a three-tier system designed to separate concerns and optimize maintainability and scalability. The presentation layer is implemented as a React.js Single Page Application that delivers core pages including Home, About, and Our Work through a mobile-first, responsive interface compliant with WCAG 2.1 AA accessibility standards. The application layer consists of a Node.js/Express backend responsible for API routing and business logic, including content management, contact form processing, media optimization, and cache management. The data layer utilizes a MongoDB NoSQL database to store page content, media metadata, partner details, and contact submissions, supplemented by cloud storage for images and videos delivered via a Content Delivery Network for performance and reliability. This layered structure also helps maintain minimal hosting costs.

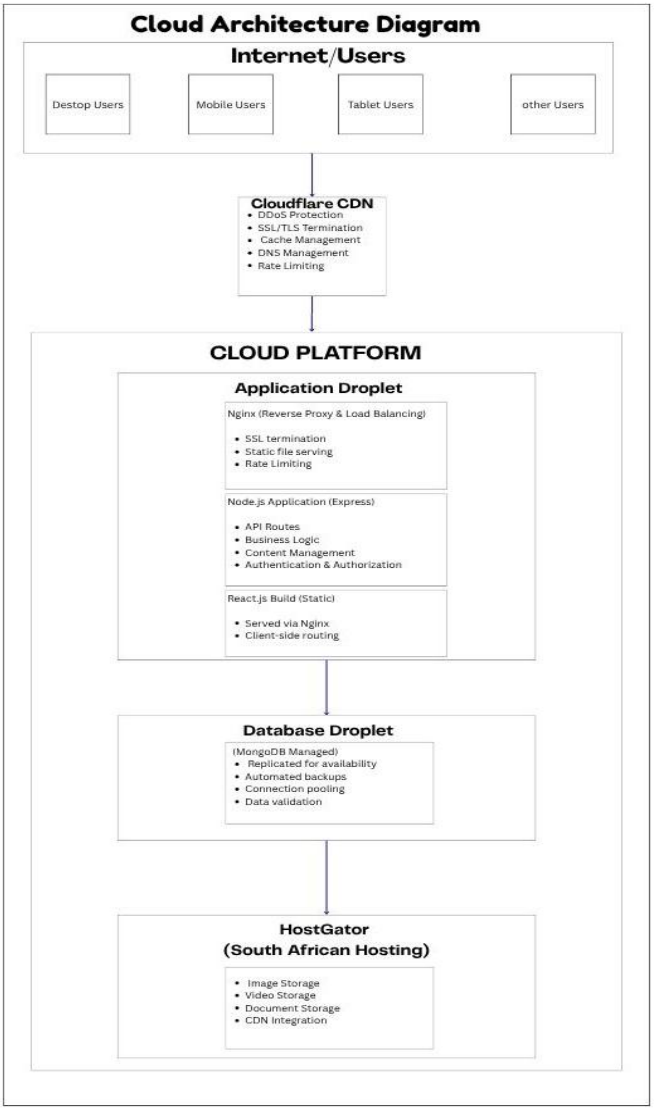
Design Pattern

1. Model-View-Controller (MVC) Pattern
    - Model: defines data structures such as Page, Partner, Event)
    - Controller: manages business logic and processes API request
    - View: Delivered by the React frontend, which consumes APIs and renders the UI.
2. Singleton Pattern
    - Used for managing database connections.
    - It maintains a single, shared connection pool to minimize resource consumption
3. Factory Pattern
    - Applied for content type creation. It instantiates the correct content handlers according to the page type.
    - Examples: PageFactory, MediaFactory.
4. Observer Pattern
    - Updates trigger notifications to observers like the cache and search index
    - Ensuring data consistency across the system
5. Repository Pattern
    - Data Access Layer
    - Abstracts data operations with a clear interface and makes it simple to swap data sources without affecting other layers.

Cloud

Selected Cloud Provider: Domains.co.za

Figure 2



[Designed on Canva]

[Made by Amahle Ndlovu]

Canva link*:* https://canva.link/vhwfbjzxfa7hn8y

Brief: Domains.co.za is an established SA-based registrar and host with 20+ years in the market, ICANN accreditation, and infrastructure in Johannesburg for low-latency performance. Plans range from R109/mo shared hosting to R209/mo VPS and R2,999/mo managed VMs, all with SSL, daily backups, and LiteSpeed. For SisterLove,

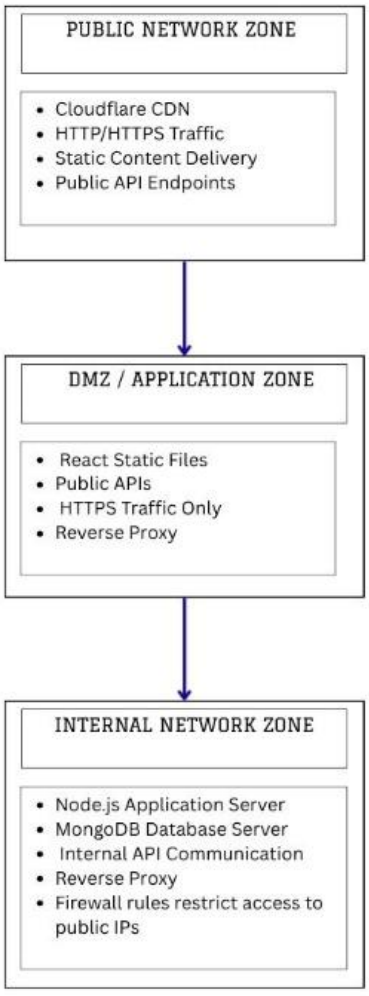
the Basic or Standard shared plans provide enough capacity for a content-driven site at an affordable price, making it a practical local option.

Why Domains.co.za

1. Local Hosting & Support: Johannesburg servers for low latency. Support via phone, chat, and email
2. Reputation & Reliability: Strong track record on service and uptime. Occasional downtime and resource quota issues reported

Network Segregation

Figure 3



[Designed on Canva]

[Made by Amahle Ndlovu]

Canva link: https://canva.link/xr9r8zfdnof364u

Brief: Network segregation divides a computer network into distinct sections, each with its own access rules. If a breach occurs, it's contained to one section, making it

harder for threats to spread elsewhere.

9.1.13. SECURITY

Security was a central design consideration throughout development, both because the system will be publicly accessible and because it collects personal information from donors, partners and volunteers, and stores it in a SQL Server database accessed through an ASP.NET MVC application. This section presents a risk analysis of the system, discusses the security principles applied, and details the specific mitigations implemented, including cross-site scripting (XSS) prevention, in line with the security principles and techniques that are in the current OWASP guidelines (OWASP, 2021) as suggested by Anthropic (2026).

9.1.13.1 Security Approach

The team applied a defence-in-depth strategy, layering controls at the network, platform, application and data levels rather than relying on any single safeguard. Three guiding principles from the module were applied throughout: least privilege (each user, role and service account is granted only the access it needs, e.g. the application's database account cannot alter schema), input validation and output encoding (all data crossing a trust boundary, from Boundary to Controller to Service to Database, is treated as untrusted until proven otherwise), and secure defaults (the system fails closed rather than open, and insecure configurations must be deliberately opted into rather than deliberately opted out of) as suggested by Anthropic (2026).

9.1.13.2 Network Security

- HTTPS/TLS is enforced site-wide, with HTTP requests redirected to HTTPS via ASP.NET's RequireHttpsAttribute/URL Rewrite rules, protecting form submissions and the administrator login (Figure 1) from interception.
- The administrator login and content management endpoints are reachable only over the authenticated area of the site; ASP.NET Identity's account lockout policy rate-limits repeated failed login attempts.
- A web application firewall (WAF), for example Azure's built-in Application Gateway WAF or an equivalent on the chosen host, filters malicious traffic before it reaches the application layer.
- The SQL Server Database is not exposed on a public port; firewall rules restrict database connections to the application server only, and the application connects using a least-privilege service account rather than a database administrator account. as suggested by Anthropic (2026).

9.1.14. DEVOP

INTRODUCTION

DevOps is a software development model that combines software development (Dev) with IT operations (Ops) to improve collaboration, automate procedures, and increase productivity. DevOps aims to shorten the software development lifecycle while maintaining reliable and high-quality software releases. By integrating development, testing, and deployment into a continuous process, teams may identify issues early, reduce manual work, and ensure the timely and dependable delivery of new features. (IBM,2024)

For this project, the team applied DevOps techniques to improve collaboration, maintain code quality, and speed up the software development process. GitHub was used for collaboration and version control, and GitHub Actions was selected to automate the build and testing processes as part of the Continuous Integration (CI) pipeline. (GitHub, 2026)

Version control

The process of monitoring and controlling modifications made to source code over time is known as version control. It enables developers to work on several project components concurrently without erasing each other's contributions. Additionally, version control offers a comprehensive history of all code modifications, enabling the ability to go back to earlier iterations in the event of mistakes. (Chacon and Straub, 2026)

The team used GitHub as the remote repository and Git as the version control system. GitHub offered a single platform for team members to store, manage, and distribute all project files. In order to maintain a clear and understandable project history, each code change was committed with a descriptive commit note. (GitHub, 2026)

The Importants of Version Control

Version control provides several benefits to the development team. It maintains a complete history of project changes, allowing developers to track modifications and restore previous versions when necessary. It also enables multiple developers to work on different parts of the project simultaneously by using branches, which helps reduce conflicts and supports organised development (Chacon and Straub, 2026). GitHub further supports collaboration by allowing developers to create Pull Requests, review code changes, discuss contributions, and safely integrate approved changes into shared branches (GitHub, 2026b). Overall, version control improves collaboration, traceability, and the reliability of the software development process.

Benefits of Version Control

- Maintains a complete history of project changes.
- Enables multiple developers to work simultaneously.
- Reduces the risk of code conflicts.
- Supports collaboration through Pull Requests.
- Makes it easier to restore previous versions if required.
- Provides secure cloud-based storage for the project repository.

(GitHub, 2026)

GitHub Collaboration

GitHub served as the primary collaboration platform throughout the project. Each developer cloned the repository, worked on their assigned tasks in separate branches, and submitted their work through Pull Requests. Before any changes were merged into the shared codebase, team members reviewed the code to ensure that it met project standards and did not introduce errors.

Using GitHub encouraged effective teamwork by allowing developers to discuss code changes, review contributions, and resolve merge conflicts before integrating new functionality into the project. (GitHub, 2026)

Branching Strategy

The project followed a **Feature Branch Workflow**, which separates new development from the stable version of the application.

The branching strategy consisted of three main branch types:

- **Main Branch:** Stores the stable and production-ready version of the application.
- **Develop Branch:** Used to integrate completed features before releasing them to the main branch.
- **Feature Branches:** Individual branches created for each new feature or bug fix.

Each developer created a dedicated feature branch for their assigned tasks. After completing development and testing, a Pull Request was submitted for review. Once approved, the feature branch was merged into the **develop** branch. When all features had been successfully integrated and tested, the **develop** branch was merged into the **main** branch for release. (Chacon and Straub, 2026)

Figure 1

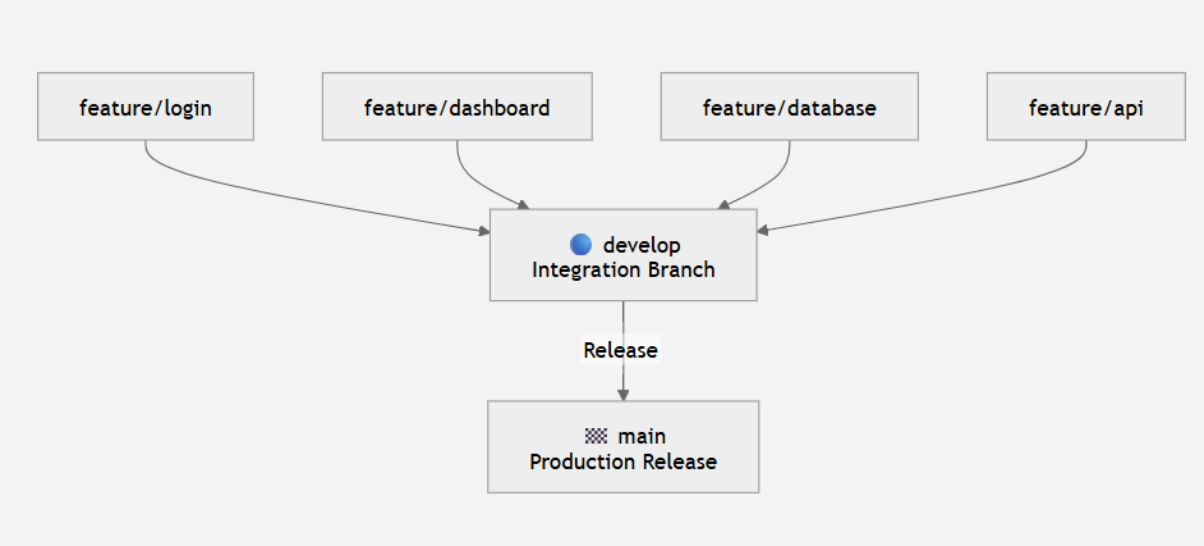


Figure 1 – Git Branching Strategy

Git Branching Strategy.drawio - draw.io

Development Workflow

The development workflow ensured that code changes followed a consistent process before being added to the final application. Developers created feature branches, developed and tested their changes, and pushed them to GitHub through Pull Requests for review. GitHub Actions automatically built and tested the changes before they were merged, helping maintain code quality and reduce errors (GitHub, 2026b; GitHub, 2026c).

The workflow followed these steps:

1. Pull the latest changes from the GitHub repository.
2. Create a new feature branch from the develop branch.
3. Develop and test the assigned functionality locally.
4. Commit changes using meaningful commit messages.
5. Push the feature branch to GitHub.
6. Create a Pull Request for code review.
7. GitHub Actions automatically builds and tests the application.
8. If all automated checks pass, the Pull Request is reviewed and approved.
9. Merge the feature branch into the develop branch.
10. Merge the develop branch into the main branch when preparing a new release.

This structured workflow ensured that only tested and reviewed code was incorporated into the production version of the application. (GitHub, 2026)

Figure 2



Figure 2 – Development Workflow

[development workflow.drawio - draw.io]

(GitHub, 2026)

Continuous Integration (CI)

Continuous Integration (CI) is a DevOps practice that regularly integrates code changes into a shared repository. It helps developers identify errors early by automatically building and testing new changes before they are merged **(Red Hat, 2026)**. In this project, GitHub Actions was used to automate the CI process. When a developer pushes code or creates a Pull Request, the configured workflow builds the application and runs automated tests. If the checks pass, the changes can continue to the review and merge process; if they fail, developers can fix the problems before the code is added to the main project **(GitHub, 2026c)**.

Continuous Integration is implemented by the project using GitHub Actions. Every time code changes are committed to the repository, GitHub Actions automatically carries out pre-established actions. (Red Hat, 2026)

The following automated tasks are carried out by the CI pipeline:

examines the most recent source code version.

creates the necessary setting for development.

reinstates the dependencies of the project.

assembles the application.

carries out automated testing.

reports test results and build status.

This automation lowers manual labour, finds issues early, and enhances the software's overall quality. (GitHub, 2026)

GitHub Actions Pipeline

GitHub's built-in automation tool, GitHub Actions, lets developers design workflows for automatically creating, testing, and validating apps. (GitHub, 2026)

A GitHub Actions pipeline is an automated workflow that builds, tests and validates an application when configured events, such as a code push or Pull Request, occur. It allows developers to automate parts of the software development process instead of performing these tasks manually **(**GitHub, 2026c).

Importance:

The pipeline is important because it detects errors early, ensures that the application builds correctly, and checks that automated tests pass before changes are merged. This reduces manual work, improves code quality, and helps prevent faulty code from reaching the main branch (GitHub, 2026a).

GitHub Actions initiates the CI workflow whenever a developer pushes code or submits a Pull Request. Before enabling the modifications to be merged, the pipeline confirms that the application builds properly and that all automated tests pass. (GitHub, 2026)

GitHub Actions promptly detects any pipeline failure, enabling developers to fix issues before they impact the main source.

(GitHub, 2026)

Figure 3

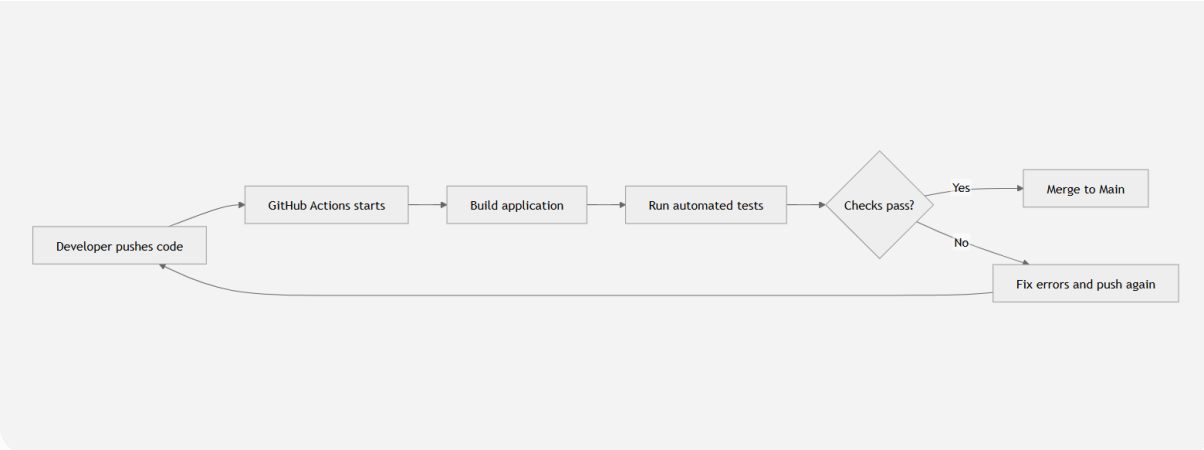


Figure 3 – GitHub Actions Pipelin

github actions.drawio - draw.io

(GitHub, 2026)

Explanation of the GitHub Actions Pipeline Diagram

1.Developer Pushes Code: The developer adds updated or new code to the GitHub repository.
2.GitHub Actions Starts: The push initiates the GitHub Actions process automatically. The Continuous Integration (CI) process is thus initiated.
3.Build Application: GitHub Actions builds the application to ensure that there are no build errors and that the code compiles appropriately.
4.Run Automated Tests: Automated tests are carried out to verify that the application operates as intended.
5.Checks Pass? GitHub Actions verifies the build and test outcomes.

(GitHub, 2026)

Figure 3 illustrates the GitHub Actions CI pipeline. When a developer pushes code, GitHub Actions automatically starts the workflow, builds the application and runs automated tests. If all checks pass, the changes can be merged into the main branch; if

a check fails, the developer fixes the issue and pushes the changes again, allowing the pipeline to repeat.

Automated Testing

Automated Testing

Automated testing is the process of using software tools to automatically run tests on an application. Instead of developers manually checking every part of the application, predefined tests are executed automatically to identify errors and confirm that the software works correctly.

Automated testing forms an essential part of the DevOps process by ensuring that new code changes do not introduce defects into the application.

Automated testing is used to detect bugs early, check that new changes do not break existing functionality, and improve the overall reliability of the application. In this project, automated testing is included in the GitHub Actions pipeline, where tests are automatically performed after the configured workflow is triggered. This provides developers with quick feedback and helps prevent faulty or unstable code from being merged into the project (GitHub, 2026a).

Within the GitHub Actions pipeline, automated tests are executed whenever changes are pushed to the repository. These tests verify that the project builds correctly, dependencies are resolved, and existing functionality continues to work as expected. (GitHub, 2026)

The automated testing process provides several benefits:

- Detects defects early in the development lifecycle.
- Prevents unstable code from being merged into shared branches.
- Improves software quality and reliability.
- Reduces manual testing effort.
- Provides immediate feedback to developers.

(GitHub, 2026)

Benefits of the DevOps Approach

The implementation of DevOps practices provided several advantages to the project:

- Improved collaboration among team members.
- Faster integration of new features.
- Automated build and testing processes reduced manual effort.
- Early detection of coding and integration issues.
- Improved software quality through continuous validation.
- Better version control and traceability of code changes.
- Reduced deployment risks through structured workflows.
- Increased reliability and maintainability of the application.

(IBM, 2024)

Conclusion

The adoption of DevOps practices enabled the development team to collaborate effectively, maintain a structured development process, and automate key stages of software delivery. By using GitHub for version control, implementing a feature branch workflow, and automating builds and testing with GitHub Actions, the team improved code quality, reduced integration issues, and ensured that only validated changes were incorporated into the project's main branch. These practices contributed to a more efficient and reliable software development lifecycle.

9.1.15. RUNNING COSTS

Due to limited organizational resources, the customer emphasized that the website should have low ongoing operating costs. As a result, if feasible, the suggested approach makes use of free-tier services and inexpensive shared WordPress hosting. The strategy eschews superfluous cloud infrastructure like enterprise hosting, Azure App Services, and premium SQL databases, which would dramatically raise monthly costs without adding value to a nonprofit educational website.

Proposed Infrastructure

Services	**Selected Solution**	**Reason**
Website Hosting	Hostinger	Low cost and WordPress optimised
CMSMS	WordPress	Free and open source
DatabaseDatabase	MySQL	Included in hosting plan
SSL Certificate	Lets Encrypt	Free
CDN	Cloudflare Free	Enhances performance
Forms	WPForms Lite	Free
Email Routing	Zoho Mail Free	Low cost
Backup	UpdraftPlus Free	Free
Security	Wordfence Free	Free

Estimated Monthly Running Costs

Service	Monthly Cost (ZAR)
Shared Hosting	R200
Domain	R99
SSL Certificate	Free
MySQL Database	Included
Cloudflare CDN	Free
Wordfence Security	Free
UpdraftPlus Backup	Free
WPForms Lite	Free
Email	Free
Total Estimated Monthly Cost	**R299 per month**

Annual operational cost: R3588 per year

Growth Projection (24 Months)

As the organization grows its volunteer base, donor engagement, community outreach, and collaborations, the client expects a steady increase in website traffic. It is anticipated that the chosen hosting package will adequately sustain the organization during its early stages of expansion. To handle rising traffic and storage needs in the event that an upgraded hosting plan is required, operational costs are anticipated to stay steady during the first year and then slightly rise in the second.

Year	Estimated Users	Estimated Monthly Cost (ZAR)	Estimated Annual Cost (ZAR)
Year 1	100	R299	R3588
Year 2	1000	R600	R7200

Justification:

The website doesn't handle a lot of transactions and is mainly informational. As a result, major infrastructural improvements are not anticipated in the first year. To maintain website availability and performance, an upgrade to a higher-tier hosting service can be necessary in the second year as the organization expands and draws more visitors.

Scaling Table for 12 months

User Scale	Infrastructure Tier	Expected Monthly Cost	Expected Annual Cost	Reason
100 users	Basic hosting	R299	R3588	Ideal for a low-traffic
1000 users	Entry-level cloud hosting	R599	R7188	As listings, photos, and API calls rise, more CPU, RAM, and storage are needed.
10000 users	Scalable cloud hosting	R1999	R23988	To manage increased traffic, database activity, pictures, and

				concurrent users, dedicated and scalable resources are needed.

9.1.16. CHANGE MANAGEMENT

How and why will the organisation adopt your software?

1. Strategic Rationale: SLiSA needs a professional website to:
    - Gain funder trust
    - Establish presence in the South African NGO space
    - Showcase 10+ years of impact, e.g. DREAMS programme 2016-2019
    - Strengthen competitiveness in securing funding
2. Operational Efficiency: the website aims to
    - Minimize time spent on manual communications
    - Streamline engagement with partners
    - Centralize all programme information
    - Automate the handling of contact inquiries
3. Funder Requirements: Funders typically require
    - A credible website
    - Clear, transparent programme details
    - Demonstrable impact reporting

How and why will the users adopt your software?

1. Community Engagement: Stakeholders are seeking:
    - Information on HIV prevention and support services
    - Access to programme details
    - Opportunities to participate
    - Pathways for partner collaboration
2. Trust and Credibility: Users will adopt the site because:
    - Transparent information builds trust
    - A local identity fosters community connections
    - A proven track record demonstrates reliability
3. Accessibility: Adoption drivers:
    - Works on any device, mobile-first
    - Free to use, no registration needed
    - Contact details are easy to access

What is your strategy to gain adoption from both the organisation and the users?

Adoption Plan:

1. Leadership:
    - Position the website as a strategic asset
    - Demonstrate alignment with organizational goals
    - Highlight its role in supporting funding efforts
2. Staff:
    - Provide CMS training
    - Prove updates are quick and simple
    - Showcase efficiency features
3. Workflow Integration
    - Connect the website to partner communications
    - Use the website to publish programme updates
    - Link the website to stakeholder engagement activities
4. Success Metrics:
    - Website visits and traffic
    - Contact form submissions
    - Partner interaction and engagement

Adoption & Launch Plan

1. Promotion
    - Launch via: Social media
        - Partner networks
        - Email newsletters
        - Community events
2. Content:
    - Timely, useful information
    - Impact stories and testimonials
    - Visual proof through photos and videos
    - Clear partnership opportunities
3. Engagement

- Strong CTAs on every page
- Simple ways to contact us
- Volunteer sign-up options
- Partner inquiry forms

4. Feedback:
    - Monitor feedback via contact forms
    - Analyze top-viewed content
    - Get partner input through surveys

System Handover Plan

Figure 4

Phase 1	▪ Set up hosting and domains ▪ Launch initial content ▪ Provision SLiSA admin accounts ▪ Produce documentation
Phase 2	Train the team on: ▪ Using the CMS ▪ Uploading media ▪ Managing contact forms ▪ Reading analytics Provide the following documentation: ▪ Admin User Guide ▪ Content Management Guide ▪ Troubleshooting Guide
Phase 3	▪ Hands-on help with updates ▪ Rapid bug fixes ▪ Assistance with initial content ▪ Gather early user feedback
Phase 4	▪ Client testing and approval ▪ Final content check ▪ Performance testing ▪ Project sign-off
Phase 5	Maintenance Plan ▪ 30 days critical issue support ▪ Monthly security updates ▪ Quarterly analytics reviews

Brief: The handover plan ensures a structured transition of the SisterLove South Africa website from the development team to SLiSA, supporting operational readiness and

long-term sustainability. Over 8 weeks, the process moves through environment setup, domain configuration, documentation delivery, and staff training on CMS, media, and analytics. During the transition period, SLiSA gradually assumes control with developer shadow support, while the team provides bug fixes and weekly check-ins. The project concludes with final acceptance testing, leadership review, and formal sign-off. Post-launch, SLiSA receives a 30-day warranty for critical issues, one year of free monthly security updates, and quarterly analytics reviews to guide content strategy. This phased approach ensures knowledge transfer, system stability, and client independence while minimizing risks.

Transition Strategy

As SLiSA has limited digital operations at present:

1. Communication
    - Announce the launch of the new website to all stakeholders
    - Distribute website links via email and social media channels
    - Share the website with partner organizations
2. Training Support
    - Video Tutorials: Provide step-by-step video guides
    - Quick Reference Guides: Develop easy-to-use reference materials
    - One-on-One Support: Offer individual training sessions where required
3. Migration
    - Migrate key documents to the website
    - Organize media assets (photos, videos) for easy access
    - Develop structured programme information
4. Support
    - Dedicated support contact
    - Weekly check-ins for month 1
    - Ticket system for issues

Training Plan:

Admin training

Figure 5

Module	Length	Focus topic
Introduction	30min	Overview and Access Setup
Managing Website Content	60min	Create/edit pages, upload media, update programmes
Contact Administration & Support	30min	Manage inquiries and respond to contacts

Analytics & Insights	30min	Google Analytics Dashboard & Performance Reporting
Open Discussion & Q&A	30min	Hands-on Training and Q&A

Brief: The training plan prepares the SLiSA team to independently manage the SisterLove South Africa website. Delivered over two weeks, it consists of four modules and supervised practice using a blended approach of live sessions, workshops, videos, guides, and peer learning. The programme covers system access, content and media management, partner updates, engagement tracking, analytics, and maintenance. Designed for consultants, volunteers, and content administrators with diverse technical skills, the training ensures participants can publish content, manage forms, monitor performance, and resolve common issues for sustainable site operations.

User Training Materials

- Video Tutorials – Quick lessons, 5-10 minutes each
- Downloadable User Guide – PDF
- FAQs
- Get Help – Support contacts

Training Schedule:

Figure 6

Week	**Activity**	**Target Participants**
Week 1	System overview & initial setup	SLiSA leadership
Week 2	Content management training	Team
Week 3	Practical hands-on session	Content managers
Week 4	Advanced system features	Interested team members

9.1.17. APPENDICES

The Appendices below are structured to hold material that is referenced in, but not reproduced in full within, the body of this report.

Appendix A: Website Development Brief and Correspondence

Appendix B: List of Figures

- Figure 1: Administrator Login Sequence Diagram
- Figure 2: Volunteer Registration Sequence Diagram
- Figure 3: Funding Enquiry Sequence Diagram
- Figure 4: Administrator Content Management Sequence Diagram
- Figure 5: Volunteer Registration State Diagram
- Figure 6: Website Content Lifecycle State Diagram
- Figure 7: Deployment Architecture Diagram

Declaration of Authenticity

This report is our own, individual work, submitted to the SLiSA website development group project.

We acknowledge that copying another person's work, in whole or in part, and presenting it as our own is a form of academic dishonesty, and we declare that this is our own work in our own words, with acknowledgement given to the sources we have used, including internet sources and any generative AI tools, as set out in the AI Declaration

We have not used another current or previous student's work and submitted it as our own, and we have not allowed, and will not allow, any other person to copy this work and submit it as their own.

Where this work forms part of a group submission, we confirm that the content above reflects our own individually authored contribution to the group project, and that any group-authored or shared material (e.g. diagrams, brief extracts) has been acknowledged as such within the text.

AI DECLARATION

Generative AI Claude SONNET 5 Medium was used during the preparation of some sections in this report for the following purposes only:

- Proofreading and language editing of drafted text for grammar, spelling and clarity.
- Formatting assistance, such as structuring headings, tables and the reference list according to the IIE Harvard-Anglia style.
- Suggesting phrasing for transitional sentences between sections.

Generative AI was not used to generate the technical analysis, domain model, sequence/state diagrams, risk assessments, or security recommendations contained in this report; these reflect the author's own understanding of the SLiSA brief and the module content. All AI-suggested edits were reviewed, verified, and, where necessary, corrected by the author before inclusion, and the author takes full responsibility for the accuracy and originality of the final content.

REFERENCE

Anthropic, 2026. Claude [large language model]. [online] Available at : <https://claude.ai> [Accessed 03 August 2026]

Graça, H., 2017. Medium [Online]. Available at: https://medium.com/the-software-architecture-chronicles/layered-architecture-a6371b945ceb

[Accessed by 30 July 2026].

GeeksforGeeks, 2025. [Online]. Available at: https://www.geeksforgeeks.org/system-design/design-patterns-architecture/

[Accessed by 31 July 2026].

GeeksforGeeks, 2026. Mircoservices, [Online]. Available at: https://www.geeksforgeeks.org/system-design/microservices/

[Accessed 1 August 2026].

GeeksforGeeks, 2026. Serveless Architecture, 23 June 2026. [Online]. Available at: https://www.geeksforgeeks.org/system-design/serverless-architectures/

[Accessed 1 August 2026].

GeeksforGeeks, 2026. MVC Architecture – System design, 22 June 2026. [Online]. Available at: https://www.geeksforgeeks.org/system-design/mvc-architecture-system-design/

[1 August 2026].

GeeksforGeeks, 2026. Singleton Method Design Pattern, 5 May 2026. [Online]. Available at: https://www.geeksforgeeks.org/system-design/singleton-design-pattern/

[Accessed 1 August 2026].

GeeksforGeeks, 2026. Factory method Design Pattern, 4 May 2026. [Online]. Available at: https://www.geeksforgeeks.org/system-design/factory-method-for-designing-pattern/

[Accessed 1 August 2026].

GeeksforGeeks, 2026. Observer Design Pattern, 7 June 2026 [Online]. Available at: https://www.geeksforgeeks.org/system-design/observer-pattern-set-1-introduction/

[Accessed 1 August 2026].

GeeksforGeeks, 2026. Repository Design Pattern, 13 May 2026. [Online]. Available at: https://www.geeksforgeeks.org/system-design/repository-design-pattern/

[Accessed 1 August 2026].

GeeksforGeeks, 2026. Unified Modeling Language (UML) Diagram, 16 April 2026. [Online]. Available at: https://www.geeksforgeeks.org/system-design/unified-modeling-language-uml-introduction/

[Accessed 16 August 2026].

GitHub (2026a). *About Continuous Integration with GitHub Actions.* GitHub Docs. [online] Available at: https://docs.github.com/en/actions/use-cases-and-examples/building-and-testing/building-and-testing-nodejs

[Accessed 17 August 2026].

GitHub (2026b). *About Pull Requests.* GitHub Docs. [online] Available at: https://docs.github.com/en/pull-requests/get-started/about-pull-requests

[Accessed 17 August 2026].

GitHub (2026c). *Understanding GitHub Actions.* GitHub Docs. [online] Available at: https://docs.github.com/en/actions/get-started/understand-github-actions

[Accessed 17 August 2026].

Graça, H. (2017). *Layered Architecture.* Medium. [online] Available at: https://medium.com/the-software-architecture-chronicles/layered-architecture-a6371b945ceb

[Accessed 30 July 2026].

IBM (2024). *What is DevOps?* IBM. [online] Available at: https://www.ibm.com/think/topics/devops

[Accessed 17 August 2026].

Mothapo, N., 2026. SisterLove Internation South Africa website development brief and website content [personal communication]. SisterLove, Inc

OWASP, 2021. A03:2021-Injection. [online] Avaible at: <https://owasp.org/Top10/2021/A03_2021-INJECTION/> [Accessed 04 August 2026]

Red Hat (2026). *What is Continuous Integration (CI)?* [online] Available at: https://www.redhat.com/en/topics/devops/what-is-continuous-integration

[Accessed 17 August 2026].

SamMonoRT (2023). Transactions - EF core. [online] MicrosoftLearn. Available at: https://learn.microsoft.com/en-us/ef/core/saving/transactions?utm

[Accessed 16 August 2026].

wadepickett (2026a). Introduction to identity on asp.net core. [online] MicrosoftLearn. Available at: https://learn.microsoft.com/en-us/aspnet/core/security/authentication/identity?view=aspnetcore-10.0&utm&tabs=visual-studio

[Accessed 16 August 2026].

Wadepickett (2026b). Role-based authorization in ASP.NET core MVC. [online] MicrosoftLearn. Available at: https://learn.microsoft.com/en-us/aspnet/core/mvc/security/authorization/roles?view=aspnetcore-10.0&utm [Accessed 16 August 2026].

WilliamDAssafMSFT (2026). Primary and foreign key constraints - SQL server. [online] MicrosoftLearn. Available at: https://learn.microsoft.com/en-us/sql/relational-databases/tables/primary-and-foreign-key-constraints?view=sql-server-ver16&utm [Accessed 16 August 2026].

WordPress. 2026. Roles and Capabilities [online] Available at: < https://wordpress.org/documentation/article/roles-and-capabilities/ > [Accessed 12 August 2026]

WordPress. 2026. Users [online] Available at: < https://developer.wordpress.org/plugins/users/ > [Accessed 12 August 2026]